

Documentación de Código Fuente

Generado el: 2025-12-11 12:55

1. Estructura de Directorios

Estructura del Proyecto:

```
./
  streamlit_app.py
  README.md
  CONTRIBUTING.md
  resources/
    manual_images/
  modules/
    convert_images.py
    __init__.py
    transformer.py
    correction.py
    xml_html.py
  views/
    creditos.py
    transformador.py
    manual_usuario.py
    documentacion.py
```

2. Contenido de Archivos

Código Fuente - Transformador XML JATS

Archivo: CONTRIBUTING.md

Contribuir al Proyecto

¡Gracias por tu interés en mejorar el Transformador XML JATS! Este documento describe las normas y mejores prácticas para contribuir al código.

Configuración del Entorno de Desarrollo

1. **Clonar el repositorio**:

```
```bash
git clone <URL_DEL_REPOSITORIO>
cd <NOMBRE_DEL_DIR>
```
```

2. **Crear un entorno virtual** (Python 3.9+ recomendado):

```
```bash
python -m venv .venv
source .venv/bin/activate # En Windows: .venv\Scripts\activate
```
```

3. **Instalar dependencias**:

```
```bash
pip install -r requirements.txt
```
```

4. **Configurar Variables de Entorno**:

Asegúrate de tener acceso a la CLI de Gemini o configurar la variable `GEMINI\_API\_KEY` si el código lo requiere directamente.

Estilo de Código y Normas

Para mantener la calidad y legibilidad del código, exigimos el cumplimiento de las siguientes normas en todos los Pull Requests:

1. Convenciones Generales

- Seguimos **PEP 8**.
- Longitud de línea máxima sugerida: **100 caracteres**.
- Usa **snake_case** para funciones y variables, y **PascalCase** para clases.

2. Type Hints (Tipado Estático)

Todas las funciones y métodos **deben** tener anotaciones de tipo (Type Hints) completas para argumentos y valores de retorno.

Correcto:

```
```python
def procesar_texto(entrada: str, opciones: Dict[str, bool]) -> List[str]:
 ...
```
```

Incorrecto:

```
```python
def procesar_texto(entrada, opciones):
 ...
```
```

Código Fuente - Transformador XML JATS

...

3. Docstrings (Estilo Google)

Todas las funciones, clases y módulos públicos deben tener docstrings siguiendo el **Google Python Style Guide**.

Ejemplo:

```
```python
def funcion_ejemplo(arg1: int, arg2: str) -> bool:
 """Realiza una operación de ejemplo.

 Explica brevemente qué hace la función. Puede tener múltiples líneas
 si es complejo.

 Args:
 arg1 (int): El primer argumento, que representa X.
 arg2 (str): El segundo argumento, que representa Y.

 Returns:
 bool: True si la operación fue exitosa, False en caso contrario.

 Raises:
 ValueError: Si arg1 es negativo.
 """
 ...
```
```

Flujo de Trabajo

1. **Reportar Problemas**: Abre un Issue antes de comenzar trabajos grandes.
2. **Ramas (Branches)**: Crea ramas descriptivas, ej. ``feature/validacion-xml`` o ``fix/error-parseo``.
3. **Pull Requests**:
 - Describe claramente tus cambios.
 - Asegúrate de que el código pase las validaciones locales.
 - Adjunta capturas de pantalla si cambiaste la interfaz gráfica.

Contacto

Para dudas técnicas, contacta a: [\[cristian.carreno@uv.cl\]\(mailto:cristian.carreno@uv.cl\)](mailto:cristian.carreno@uv.cl)
Universidad de Valparaíso.

Código Fuente - Transformador XML JATS

Archivo: README.md

Transformador XML JATS (JATS XML Transformer) - v0.5 (Beta)

Una herramienta avanzada impulsada por Inteligencia Artificial para convertir documentos de Word (`.docx`) a formato **JATS XML** validado, diseñada específicamente para el flujo editorial de revistas científicas.

Esta aplicación automatiza el proceso de etiquetado semántico, extracción de tablas e imágenes (en desarrollo), y validación contra el estándar NLM/NCBI Journal Publishing DTD v1.3.

? Características Principales

- **Conversión Inteligente**: Utiliza LLMs (Google Gemini) para interpretar la estructura lógica del documento.
- **Asistente de Corrección Interactivo**: Chatbot integrado que ayuda a solucionar errores de validación DTD, permitiendo ediciones precisas y regeneración del XML.
- **Navegación Fluida**: Interfaz intuitiva con pestañas y botones de navegación automática entre pasos.
- **HTML Autocontenido**: Genera archivos HTML con el logo incrustado (Base64), listos para publicar sin dependencias externas.
- **Validación Integrada**: Valida contra JATS 1.3 y ofrece sugerencias de corrección en tiempo real.
- **Conversión Inteligente**: Utiliza LLMs (Google Gemini) para interpretar la estructura lógica del documento y generar etiquetas JATS precisas.
- **Extracción de Contenido**: Detecta y extrae imágenes y tablas automáticamente desde el archivo Word.
- **Validación Integrada**: Valida el XML generado contra el DTD oficial JATS 1.3 con MathML3.
- **Interfaz Dual**:
 - **CLI (Línea de Comandos)**: Para automatización y procesamiento por lotes.
 - **Web UI (Streamlit)**: Interfaz gráfica amigable para arrastrar y soltar archivos, editar contenido extraído y previsualizar resultados.
- **Conversión a HTML**: Genera una vista previa en HTML del artículo para su revisión inmediata.

?? Requisitos del Sistema

- Windows, macOS o Linux.
- [Python 3.9+](https://www.python.org/downloads/)
- Una clave de API de Google Gemini (Google AI Studio).

? Instalación

1. **Clonar el repositorio** (si aplica) o descargar el código fuente.
2. **Crear un entorno virtual** (recomendado):

```
```bash
python -m venv .venv
Activar:
Windows: .venv\Scripts\activate
macOS/Linux: source .venv/bin/activate
```
```

3. **Instalar dependencias**:

```
```bash
pip install -r requirements.txt
```
```

Nota: Si no tienes un `requirements.txt`, las dependencias principales son: `python-docx`, `lxml`, `streamlit`, `google-generativeai` (o la herramienta CLI correspondiente).

4. **Configurar la API Key de Gemini**:
Asegúrate de que la CLI de `gemini` esté configurada en tu PATH o system environment variables.

Código Fuente - Transformador XML JATS

```
```bash
export GEMINI_API_KEY="tu_clave_aqui"
```

## ? Uso

### Interfaz Web (Recomendado)

La interfaz gráfica es la forma más fácil de usar la herramienta.

1. Ejecuta la aplicación:

```bash
streamlit run streamlit_app.py
```

2. Abre tu navegador en la URL mostrada (usualmente `http://localhost:8501`).
3. Sube tu archivo `.docx`.
4. Revisa el texto extraído, genera el XML, valida y descarga los resultados.

### Línea de Comandos (CLI)

Para usuarios avanzados que deseen integrar la herramienta en scripts.

**Transformación (Word -> XML):**

```bash
python -m modules.transformer entrada.docx salida.xml
```

**Conversión (XML -> HTML):**

```bash
python -m modules.xml_html entrada.xml salida.html
```

## ? Estructura del Proyecto

- `streamlit_app.py`: Punto de entrada de la aplicación web (Streamlit UI).
- `modules/`:
  - `transformer.py`: Núcleo de la lógica de conversión. Maneja la lectura del Word, construcción del prompt para IA y validación XML.
  - `correction.py`: Módulo para la corrección asistida por IA.
  - `xml_html.py`: Utilidad para convertir el XML JATS resultante a HTML visualizable.
- `views/`: Vistas de la interfaz gráfica.
- `JATS-Publishing-1-3-MathML3-DTD/`: Archivos DTD locales para validación offline (se descargan si no existen).
- `imagenes_extraidas/`: Directorio temporal donde se guardan las imágenes extraídas del documento Word.

## ? Créditos

Desarrollado por **Cristian Carreño León**\
Escuela de Obstetricia y Puericultura\
Facultad de Medicina\
Universidad de Valparaíso, Chile.

Desarrollado para la **Universidad de Valparaíso** con el objetivo de optimizar los procesos de publicación científica.

## ? Historial de Versiones (Changelog)

### v0.5 (Beta) - Versión Actual
```

Código Fuente - Transformador XML JATS

- ****Mejoras de UI****: Navegación por pestañas con botones de avance automático.
- ****Corrección Interactiva****: Nuevo chatbot que permite dialogar con la IA para resolver errores de validación. Implementación de una "Caja de Sugerencias" para revisar y aplicar cambios al XML de forma segura.
- ****Logo Embedded****: El logo de la revista ahora se incrusta como Base64 en el HTML generado, eliminando la dependencia de carpetas locales.
- ****Optimización****: Eliminación de animaciones intrusivas y mejora en la legibilidad del chat en modo oscuro.

v0.1 - v0.4 (Alpha)

- Inicio del proyecto.
- Configuración de Gemini CLI.
- Extracción básica de DOCX.
- Primera implementación de transformación a XML JATS.

Código Fuente - Transformador XML JATS

Archivo: modules/__init__.py

Archivo: modules/convert_images.py

```
from PIL import Image
import os
import glob

image_dir = "resources/manual_images"
# Get the numbered images the user added
images = glob.glob(os.path.join(image_dir, "0*.png"))
# Add the logo
images.append("resources/UV_color.png")

print(f"Found {len(images)} images to process: {images}")

for img_path in images:
    if not os.path.exists(img_path):
        print(f"Image not found: {img_path}")
        continue

    try:
        with Image.open(img_path) as img:
            # Convert to RGB
            img = img.convert('RGB')
            # Save it back without interlacing
            img.save(img_path, "PNG", optimize=False, compress_level=0)
            print(f"Converted and saved: {img_path}")
    except Exception as e:
        print(f"Failed to convert {img_path}: {e}")
```


Código Fuente - Transformador XML JATS

Archivo: modules/correction.py

```
# -*- coding: utf-8 -*-

"""Módulo para la corrección asistida por IA de archivos XML JATS.

Este módulo permite enviar un XML inválido junto con sus errores de validación
y feedback del usuario a un modelo de lenguaje (Gemini) para obtener una versión corregida.
"""

import subprocess
import sys
import time
from typing import Dict, Any, List

from .transformer import invocar_gemini_cli, WORKSPACE_ROOT

def construir_prompt_analisis_inicial(xml_content: str, validation_errors: List[str]) -> str:
    """Construye el prompt para el análisis inicial de errores."""
    errores_str = "\n".join(f"- {err}" for err in validation_errors)

    prompt = f"""
    Actúa como un validador experto de XML JATS 1.3.

    FINALIDAD:
    Determinar si los errores de validación se deben a DATOS FALTANTES (que requieren input del usuario) o a
    ERRORES ESTRUCTURALES/SINTÁCTICOS (que puedes corregir tú mismo).

    ERRORES REPORTADOS:
    {errores_str}

    XML ACTUAL:
    ```xml
 {xml_content}
    ```

    INSTRUCCIONES:
    1. Si detectas que FALTA INFORMACIÓN ESPECÍFICA necesaria para validar (ej: falta apellido del autor, falta
    año de publicación, falta título de la revista, faltan referencias), TU RESPUESTA DEBE SER UNA PREGUNTA CLARA
    AL USUARIO pidiendo esos datos exacta y amablemente. NO GENERES XML.

    2. Si los errores son puramente TÉCNICOS o ESTRUCTURALES (ej: tag mal anidado, atributo inválido, orden
    incorrecto de elementos conocidos) y NO necesitas información nueva:
        - Genera el XML corregido completo.
        - Envuelve el XML en un bloque: ```xml ... ```
        - Agrega una breve nota fuera del bloque explicando qué arreglaste.

    PRIORIDAD: Si hay mezcla de errores, PRIORIZA PREGUNTAR POR LOS DATOS FALTANTES antes de intentar corregir
    la estructura. No inventes datos.

    TU RESPUESTA:
    """
    return prompt

def analizar_errores_inicial(
    xml_content: str,
    validation_errors: List[str]
) -> Dict[str, Any]:
    """Realiza el análisis inicial de errores para decidir si preguntar o corregir."""
    prompt = construir_prompt_analisis_inicial(xml_content, validation_errors)
    return invocar_gemini_cli(prompt)
```

Código Fuente - Transformador XML JATS

```
def construir_prompt_correccion(
    xml_content: str,
    validation_errors: List[str],
    user_feedback: str = ""
) -> str:
    """Construye el prompt para solicitar correcciones al modelo (flujo normal)."""
    errores_str = "\n".join(f"- {err}" for err in validation_errors)

    extra_instructions = ""
    if user_feedback:
        extra_instructions = f"""
INSTRUCCIONES ADICIONALES DEL USUARIO:
"{user_feedback}"
Asegúrate de atender específicamente solicitud del usuario.
        """

    prompt = f"""
Actúa como un experto en depuración de XML JATS 1.3 interactivo.

SITUACIÓN:
Tengo un archivo XML que NO valida contra el DTD JATS 1.3 o tiene datos incompletos.

ERRORES DE VALIDACIÓN REPORTADOS O PREGUNTA DEL USUARIO:
{errores_str}

{extra_instructions}

TAREA:
1. Analiza el problema.
2. SI TIENES INFORMACIÓN SUFICIENTE para corregir con lo que el usuario ha dado:
   - Genera el XML corregido completo.
   - Envuelve el XML en un bloque de código markdown: ```xml ... ```
3. SI AÚN FALTA INFORMACIÓN CRÍTICA:
   - PREGUNTA nuevamente.

ENTRADA XML ACTUAL:
```xml
{xml_content}
```

TU RESPUESTA:
"""
    return prompt

def corregir_xml(
    xml_content: str,
    validation_errors: List[str],
    user_feedback: str = ""
) -> Dict[str, Any]:
    """Orquesta el proceso de corrección de XML usando Gemini."""
    prompt = construir_prompt_correccion(xml_content, validation_errors, user_feedback)
    return invocar_gemini_cli(prompt)
```

Código Fuente - Transformador XML JATS

Archivo: modules/transformer.py

```
# -*- coding: utf-8 -*-

"""Script para convertir documentos Word a JATS XML usando IA.

Este módulo orquesta la extracción de contenido de archivos .docx, la generación
de XML JATS mediante un modelo de lenguaje (Gemini), y la validación del resultado.
Cumple con las normas de documentación de Google.

Attributes:
    DTD_ZIP_URL (str): URL para descargar el DTD de JATS 1.3.
    WORKSPACE_ROOT (Path): Ruta base del espacio de trabajo actual.
    DTD_FILENAME (str): Nombre del archivo DTD local.
    IMAGE_OUTPUT_DIR (Path): Directorio para guardar imágenes extraídas.
"""

import os
import re
import subprocess
import sys
import time
import urllib.request
import zipfile
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Optional, Tuple, Any

from docx import Document
from lxml import etree

# --- Constantes ---
DTD_ZIP_URL = "https://public.nlm.nih.gov/projects/jats/publishing/1.3/JATS-Publishing-1-3-MathML3-DTD.zip"
WORKSPACE_ROOT = Path(__file__).resolve().parent
DTD_FILENAME = "JATS-journalpublishing1-3-mathml3.dtd"
DTD_LOCAL_FILE = WORKSPACE_ROOT / DTD_FILENAME
IMAGE_OUTPUT_DIR = WORKSPACE_ROOT / "imagenes_extraidas"

def extraer_contenido_estructurado(docx_path: str) -> Optional[str]:
    """Extrae contenido de un .docx, incluyendo texto, tablas e imágenes.

    Las imágenes se guardan en el disco y se insertan placeholders en el texto.
    Las tablas se convierten a una representación de texto con delimitadores.

    Args:
        docx_path (str): Ruta absoluta o relativa al archivo .docx.

    Returns:
        Optional[str]: El contenido extraído como texto con placeholders, o
            None si ocurre un error crítico (archivo no encontrado, etc.).
    """
    try:
        doc = Document(docx_path)
        content_parts: List[str] = []
        image_counter = 1

        # Crear directorio para imágenes si no existe
        IMAGE_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

        # Usamos un iterador para procesar elementos del cuerpo (párrafos y tablas)
```

Código Fuente - Transformador XML JATS

```
for element in doc.element.body:
    if element.tag.endswith('p'):
        # Encontrar el párrafo correspondiente en el objeto doc.paragraphs
        # Nota: Este método de búsqueda por XPath puede ser lento en docs muy grandes
        # pero es necesario para mantener el orden estricto entre tablas y texto.
        para = doc.paragraphs[int(element.xpath('count(preceding-sibling::w:p)'))]

        # Manejo de imágenes (inline shapes)
        if 'graphicData' in para._p.xml:
            for shape in para._p.xpath('.//pic:pic'):
                # Extraer la relación de la imagen
                embeds = shape.xpath('..//a:blip/@r:embed')
                if not embeds:
                    continue
                rel_id = embeds[0]
                image_part = doc.part.related_parts[rel_id]

                # Guardar la imagen
                content_type = image_part.content_type.split('/')[1]
                image_filename = f"imagen_{image_counter}.{content_type}"
                image_path = IMAGE_OUTPUT_DIR / image_filename
                with open(image_path, "wb") as f:
                    f.write(image_part.blob)

                # Añadir placeholder
                caption = para.text.strip()
                content_parts.append(f'\n[IMAGEN-PLACEHOLDER file="{image_filename}"
caption="{caption}"]\n')
                image_counter += 1
            else:
                content_parts.append(para.text)

        elif element.tag.endswith('tbl'):
            table = doc.tables[int(element.xpath('count(preceding-sibling::w:tbl)'))]
            table_content: List[str] = []
            for row in table.rows:
                row_content = [cell.text.strip().replace('\n', ' ') for cell in row.cells]
                table_content.append(" | ".join(row_content))

            # Intentar usar el párrafo anterior como caption si parece serlo
            caption = ""
            if content_parts and content_parts[-1].strip():
                caption = content_parts[-1].strip()
                content_parts[-1] = "" # Eliminarlo para no duplicar

            content_parts.append(f'\n[TABLA-PLACEHOLDER caption="{caption}" content="{';
".join(table_content)}"]\n')

            return '\n'.join(content_parts)

except FileNotFoundError:
    print(f"Error: El archivo '{docx_path}' no fue encontrado.", file=sys.stderr)
    return None
except Exception as e:
    print(f"Error al leer o procesar el archivo Word: {e}", file=sys.stderr)
    return None
```

```
def construir_prompt_avanzado(texto_articulo: str) -> str:
    """Construye un prompt de sistema detallado para la generación de JATS XML.
```

Args:

Código Fuente - Transformador XML JATS

texto_articulo (str): El texto crudo del artículo con placeholders.

Returns:

str: El prompt completo formateado para el modelo de IA.

"""

prompt = f"""

Actúa como un experto en etiquetado JATS (Journal Article Tag Suite) XML, versión 1.3, para SciELO.

TAREA:

Convierte el siguiente texto de un artículo científico, que contiene placeholders especiales para imágenes y tablas, a un archivo XML bien formado que cumpla con el estándar JATS.

INSTRUCCIONES Y MEJORES PRÁCTICAS:

1. **Estructura General:** Raíz `<article>` con `xmlns:xlink="http://www.w3.org/1999/xlink"` y `xml:lang="es"`. Debe contener `<front>`, `<body>`, y `<back>`.

2. **Sección <front>:**

* Incluye `<article-title>` y, si está disponible en el texto, `<article-id pub-id-type="doi">`.

* **IMPORTANTE** - Orden de elementos en `<contrib>`: Dentro de cada `<contrib contrib-type="author">`, debes seguir **ESTRICTAMENTE** este orden de elementos (si están presentes):

- `<contrib-id contrib-id-type="orcid">` (si existe ORCID).
- `<name>` (con `<surname>` y `<given-names>`).
- `<xref ref-type="aff" rid="affX">` (referencias a afiliaciones).
- `<xref ref-type="corresp" rid="cor1">` (si es autor de correspondencia).
- `<email>` (correo electrónico).

NO coloques `<contrib-id>` después del nombre. NO coloques `<xref>` antes del nombre. El orden es **CRÍTICO** para la validación.

* Crea un `<aff>` por cada filiación en `<contrib-group>`, con `id` (`aff1`, `aff2`, ...) y un `<label>` numérico.

* Marca al responsable de correspondencia con `corresp="yes"` en el atributo de `<contrib>`, y registra el dato completo en `<author-notes><corresp id="cor1"><email>...</email></corresp></author-notes>`.

* Mantén `<abstract>` y `<kwd-group>` como en la especificación original.

3. **Sección <body>:**

* Usa `<sec>` para secciones con un `<title>`. Los párrafos deben ir en `<p>`.

* **Referencias en el cuerpo:** Identifica citas bibliográficas en formato numérico (Vancouver: `[1]`, `[1-3]`, etc.) y en formato autor-fecha (APA: `(Apellido, 2020)`, `(Apellido & Otro, 2019)`, etc.). Normaliza la cita reemplazando el texto original por elementos `<xref ref-type="bibr" rid="ID_DE_REFERENCIA">`. El contenido textual del `<xref>` debe reflejar el estilo original (por ejemplo, `[1]` o `(Apellido, 2020)`). Cada `<xref>` debe apuntar al `id` del `<ref>` correspondiente en la sección de bibliografía.

* **Manejo de Placeholders:**

* **Imágenes:** Si encuentras `[IMAGEN-PLACEHOLDER file="..." caption="..."]`, conviértelo a la siguiente estructura JATS:

```
``xml
<fig id="f_ID_UNICO">
  <label>Figura X</label>
  <caption><p>Texto del pie de foto aquí</p></caption>
  <graphic mimetype="image" xlink:href="NOMBRE_DEL_ARCHIVO_AQUI"/>
</fig>
``
```

Reemplaza los valores correspondientes. Genera un `id` único y una `label` secuencial.

* **Tablas:** Si encuentras `[TABLA-PLACEHOLDER caption="..." content="..."]`, donde el contenido es texto delimitado por `|` para columnas y `;` para filas, conviértelo a:

```
``xml
<table-wrap id="t_ID_UNICO">
  <label>Tabla X</label>
  <caption><p>Título de la tabla aquí</p></caption>
  <table>
    <thead>
      <tr>
        <th>Encabezado 1</th>
        <th>Encabezado 2</th>
      </tr>
    </thead>
  </table>
``
```

Código Fuente - Transformador XML JATS

```
<tbody>
  <tr>
    <td>Dato fila 1, col 1</td>
    <td>Dato fila 1, col 2</td>
  </tr>
  <tr>
    <td>Dato fila 2, col 1</td>
    <td>Dato fila 2, col 2</td>
  </tr>
</tbody>
</table>
</table-wrap>
'''
```

Interpreta la primera fila del contenido como `` con ` ` y las siguientes como `` con ` `. | |

4. ****Sección <back>:**** Usa `` y `` para la bibliografía, desglosando con ``. Asegúrate de que cada `` tenga un `id` único (por ejemplo `ref1`, `ref2`, ?) y que coincida con los atributos `rid` utilizados en los `` del cuerpo. Si detectas múltiples citas que apuntan a la misma referencia, reutiliza el mismo `id`.

5. ****Reglas Finales:****

- * El XML debe ser perfectamente bien formado.
- * No incluyas explicaciones en la salida, solo el código XML.
- * Codifica caracteres especiales (`&` como `&`, etc.).

TEXTO DEL ARTÍCULO A CONVERTIR:

{texto_articulo}

"""

return prompt

```
def _should_retry_gemini(stderr: str, stdout: str) -> bool:
```

```
    """Determina si se debe reintentar la llamada a Gemini basada en la salida de error.
```

Args:

stderr (str): Salida de error estándar del proceso.

stdout (str): Salida estándar del proceso.

Returns:

bool: True si el error indica un problema transitorio (ej. rate limit).

```
    """
```

```
    combined = f"{stderr}\n{stdout}".lower()
```

```
    retry_tokens = (
```

```
        "429",
```

```
        "ratelimit",
```

```
        "resource has been exhausted",
```

```
        "resource exhausted",
```

```
        "error when talking to gemini api",
```

```
)
```

```
    return any(token in combined for token in retry_tokens)
```

```
def invocar_gemini_cli(prompt: str, max_attempts: int = 3, base_backoff: int = 20) -> Dict[str, Any]:
```

```
    """Invoca la CLI de Gemini para procesar el prompt, con lógica de reintentos.
```

Args:

prompt (str): El prompt a enviar a Gemini.

max_attempts (int, optional): Número máximo de intentos. Por defecto 3.

base_backoff (int, optional): Segundos base para esperar entre reintentos. Por defecto 20.

Returns:

Código Fuente - Transformador XML JATS

```
Dict[str, Any]: Diccionario con 'stdout', 'stderr', 'returncode', y 'elapsed'.
"""
command = ["gemini", "-p", prompt, "-o", "text"]
last_result: Dict[str, Any] = {'stdout': '', 'stderr': '', 'returncode': 1, 'elapsed': 0.0}

for attempt in range(1, max_attempts + 1):
    try:
        start = time.time()
        result = subprocess.run(
            command,
            capture_output=True,
            text=True,
            check=False,
            encoding="utf-8",
            cwd=str(WORKSPACE_ROOT),
        )
        elapsed = time.time() - start
    except FileNotFoundError:
        msg = "Error: El comando 'gemini' no se encontró. Verifica tu instalación."
        print(msg, file=sys.stderr)
        return {'stdout': '', 'stderr': msg, 'returncode': 127, 'elapsed': 0.0}
    except Exception as exc:
        msg = f"Error inesperado al invocar a Gemini: {exc}"
        print(msg, file=sys.stderr)
        return {'stdout': '', 'stderr': msg, 'returncode': 1, 'elapsed': 0.0}

    stdout = result.stdout.strip() if result.stdout else ''
    stderr = result.stderr.strip() if result.stderr else ''
    last_result = {'stdout': stdout, 'stderr': stderr, 'returncode': result.returncode, 'elapsed': elapsed}

    # Si tenemos éxito y no hay error de rate limit, retornamos
    if stdout and not _should_retry_gemini(stderr, stdout):
        return last_result

    # Detener si llegamos al máximo
    if attempt >= max_attempts:
        break

    # Si hay error de cuota, esperar y reintentar
    if _should_retry_gemini(stderr, stdout):
        wait_time = base_backoff * attempt
        print(f"Aviso: Límite de recursos en Gemini. Reintentando en {wait_time}s...", file=sys.stderr)
        time.sleep(wait_time)
        continue

    break

return last_result


def extraer_resumen_tokens(gemini_meta: Dict[str, Any]) -> Dict[str, Any]:
    """Extrae métricas de uso de tokens de la salida de Gemini.

    Args:
        gemini_meta (Dict[str, Any]): Resultado de `invocar_gemini_cli`.

    Returns:
        Dict[str, Any]: Diccionario con 'total', 'prompt_tokens', etc.
    """
    search_text = (gemini_meta.get('stderr', '') or '') + "\n" + (gemini_meta.get('stdout', '') or '')
    tokens_info: Dict[str, Any] = {
        'found': False, 'total': None, 'prompt_tokens': None,
```

Código Fuente - Transformador XML JATS

```
'completion_tokens': None, 'raw': ''
}

# Estrategia 1: Buscar bloque JSON
try:
    m = re.search(r"\{[^\}]*token[^\}]*\}", search_text)
    if m:
        tokens_info['raw'] = m.group(0)
        total = re.search(r"total[_\s-]*tokens\D*(\d+)", m.group(0), re.I)
        pt = re.search(r"prompt[_\s-]*tokens\D*(\d+)", m.group(0), re.I)
        ct = re.search(r"completion[_\s-]*tokens\D*(\d+)", m.group(0), re.I)
        if total:
            tokens_info['total'] = int(total.group(1))
        if pt:
            tokens_info['prompt_tokens'] = int(pt.group(1))
        if ct:
            tokens_info['completion_tokens'] = int(ct.group(1))
        if tokens_info['total'] or tokens_info['prompt_tokens']:
            tokens_info['found'] = True
        return tokens_info
except Exception:
    pass

# Estrategia 2: Patrones simples
total_m = re.search(r"total\s*tokens\D*(\d+)", search_text, re.I)
pt_m = re.search(r"prompt\s*tokens\D*(\d+)", search_text, re.I)
ct_m = re.search(r"completion\s*tokens\D*(\d+)", search_text, re.I)

if total_m or pt_m or ct_m:
    tokens_info['found'] = True
    tokens_info['raw'] = search_text
    if total_m:
        tokens_info['total'] = int(total_m.group(1))
    if pt_m:
        tokens_info['prompt_tokens'] = int(pt_m.group(1))
    if ct_m:
        tokens_info['completion_tokens'] = int(ct_m.group(1))
    return tokens_info

tokens_info['raw'] = search_text
return tokens_info

def validar_jats_xml(xml_content: str) -> Tuple[bool, List[str]]:
    """Valida el contenido XML contra el DTD JATS 1.3 local.

    Args:
        xml_content (str): Cadena con el XML completo.

    Returns:
        Tuple[bool, List[str]]: (es_valido, lista_de_errores).
    """
    possible_dtd_locations = [
        WORKSPACE_ROOT / DTD_FILENAME,
        WORKSPACE_ROOT / "JATS-Publishing-1-3-MathML3-DTD" / DTD_FILENAME
    ]

    dtd_path = None
    for path in possible_dtd_locations:
        if path.exists():
            dtd_path = path
            break
```


Código Fuente - Transformador XML JATS

```
if not dtd_path:
    # Intento de descarga automática si no existe
    try:
        zip_path = WORKSPACE_ROOT / "jats_dtd.zip"
        if not zip_path.exists():
            print(f"Descargando DTD JATS desde {DTD_ZIP_URL}...")
            urllib.request.urlretrieve(DTD_ZIP_URL, str(zip_path))
            print("Descarga completada.")

        print("Extrayendo DTD...")
        with zipfile.ZipFile(zip_path, 'r') as z:
            z.extractall(WORKSPACE_ROOT)

        for path in possible_dtd_locations:
            if path.exists():
                dtd_path = path
                break
    except Exception as e:
        return False, [f"Fallo crítico al descargar/extraer DTD: {e}"]

if not dtd_path:
    return False, [f"No se encontró {DTD_FILENAME} incluso después de intentar descargar."]

print(f"Usando DTD: {dtd_path}")

try:
    # Ajustar DOCTYPE para que apunte al DTD local si es necesario,
    # o asegurar que el validador lo encuentre.
    # Aquí reemplazamos cualquier DOCTYPE existente con uno que apunte al archivo local.
    xml_content_patched = re.sub(
        r'<!DOCTYPE.*?>',
        f'<!DOCTYPE article PUBLIC "-//NLM//DTD JATS (Z39.96) Journal Publishing DTD with MathML3 v1.3
20210610//EN" "{DTD_FILENAME}">',
        xml_content,
        flags=re.DOTALL
    )

    xml_bytes = xml_content_patched.encode('utf-8')
    parser = etree.XMLParser(dtd_validation=False, no_network=False)
    xml_tree = etree.fromstring(xml_bytes, parser)
    dtd = etree.DTD(str(dtd_path))

    is_valid = dtd.validate(xml_tree)
    errors = [str(err) for err in dtd.error_log.filter_from_errors()]
    return is_valid, errors

except etree.XMLSyntaxError as e:
    return False, [f"Error de sintaxis XML (mal formado): {e}"]
except Exception as e:
    return False, [f"Error inesperado durante validación: {e}"]

def guardar_salida_xml(xml_content: str, output_path: str) -> None:
    """Guarda el XML en disco con la declaración DOCTYPE correcta.

    Args:
        xml_content (str): Contenido XML.
        output_path (str): Ruta de destino.
    """
    try:
        with open(output_path, 'w', encoding='utf-8') as f:
```

Código Fuente - Transformador XML JATS

```
dtd_decl = f'<!DOCTYPE article PUBLIC "-//NLM//DTD JATS (Z39.96) Journal Publishing DTD v1.3
20210610//EN" "{DTD_FILENAME}">'
# Eliminar declaración XML duplicada si existe
if xml_content.startswith('<?xml'):
    parts = xml_content.split('>', 1)
    if len(parts) > 1:
        xml_content = parts[-1].strip()

final_content = f'<?xml version="1.0" encoding="UTF-8">\n{dtd_decl}\n{xml_content}'
f.write(final_content)
print(f"XML guardado en: {output_path}")
except Exception as e:
    print(f"Error al escribir archivo: {e}", file=sys.stderr)

def main() -> None:
    """Función principal CLI."""
    if len(sys.argv) != 3:
        print("Uso: python transformer.py <entrada.docx> <salida.xml>", file=sys.stderr)
        sys.exit(1)

    input_path, output_path = sys.argv[1], sys.argv[2]
    base_name = os.path.splitext(output_path)[0]
    log_path = f"{base_name}.txt"

    def log(msg: str, error: bool = False) -> None:
        """Helper para logging a archivo y consola."""
        ts = datetime.now().isoformat(sep=' ', timespec='seconds')
        line = f"[{ts}] {msg}\n"
        try:
            with open(log_path, 'a', encoding='utf-8') as lf:
                lf.write(line)
        except IOError:
            pass # Ignorar errores de log

        if error:
            print(msg, file=sys.stderr)
        else:
            print(msg)

    log(f"Iniciando conversión: {input_path} -> {output_path}")

    # 1. Extracción
    log("Paso 1: Extrayendo contenido...")
    structured_text = extraer_contenido_estructurado(input_path)
    if not structured_text:
        log("Falló la extracción.", error=True)
        sys.exit(1)

    # 2. Construcción de Prompt
    prompt = construir_prompt_avanzado(structured_text)

    # 3. Invocación IA
    log("Paso 2: Consultando a Gemini AI...")
    gemini_result = invocar_gemini_cli(prompt)
    if gemini_result.get('returncode') != 0 or not gemini_result.get('stdout'):
        log("Error en Gemini AI.", error=True)
        log(f"Stderr: {gemini_result.get('stderr')}", error=True)
        sys.exit(1)

    generated_xml = gemini_result.get('stdout', '')
```

Código Fuente - Transformador XML JATS

```
# 4. Limpieza
match = re.search(r"```\xml\s*(.*?)\s*```", generated_xml, re.DOTALL)
if match:
    generated_xml = match.group(1)
else:
    # Fallback para encontrar inicio de XML
    match_xml = re.search(r"(<\?xml|<!DOCTYPE|<article).*", generated_xml, re.DOTALL)
    if match_xml:
        generated_xml = match_xml.group(0)
        if generated_xml.endswith("```"):
            generated_xml = generated_xml[:-3]

generated_xml = generated_xml.strip()

# 5. Validación
log("Paso 3: Validando XML...")
is_valid, errors = validar_jats_xml(generated_xml)
if is_valid:
    log("XML Válido según DTD.")
else:
    log("XML Inválido (se guardará igual para revisión).", error=True)
    for err in errors:
        log(f"- {err}", error=True)

# 6. Guardado
guardar_salida_xml(generated_xml, output_path)
log("Proceso terminado.")

if __name__ == "__main__":
    main()
```

Código Fuente - Transformador XML JATS

Archivo: modules/xml_html.py

```
from __future__ import annotations

"""Convertidor de JATS XML a HTML.

Este módulo se encarga de parsear archivos XML válidos bajo el esquema JATS 1.3
y generar una representación HTML5 moderna, responsiva y estilizada.
Incluye funciones para extracción de metadatos, renderizado de secciones,
y manejo de figuras y tablas.
"""

import argparse
import json
import re
import sys
from dataclasses import dataclass
from html import escape
from pathlib import Path
from typing import Any, Dict, Iterable, List, Tuple

from lxml import etree

_XML_ID = "{http://www.w3.org/XML/1998/namespace}id"
_XLINK_HREF = "{http://www.w3.org/1999/xlink}href"
_SANITIZE_ENTITY_RE = re.compile(r"&(?![a-zA-Z]+;|#\d+;|#[0-9a-fA-F]+;);")

def _clean_whitespace(text: str) -> str:
    """Normaliza los espacios en blanco de una cadena.

    Reemplaza múltiples espacios, tabulaciones y saltos de línea por un único espacio
    y elimina espacios al inicio y final.

    Args:
        text (str): Texto a limpiar.

    Returns:
        str: Texto limpio y normalizado.
    """
    return re.sub(r"\s+", " ", text or "").strip()

def _node_text(node: etree._Element) -> str:
    """Extrae todo el texto de un nodo XML y sus descendientes.

    Args:
        node (etree._Element): El nodo raíz desde el cual extraer texto.

    Returns:
        str: Todo el texto contenido en el nodo, concatenado y normalizado.
    """
    parts: List[str] = []
    if node.text:
        parts.append(node.text)
    for child in node:
        parts.append(_node_text(child))
        if child.tail:
            parts.append(_clean_whitespace(child.tail))
```

Código Fuente - Transformador XML JATS

```
        parts.append(child.tail)
    return _clean_whitespace("".join(parts))

def _inline_children_html(node: etree._Element) -> str:
    """Renderiza los hijos de un nodo como HTML inline, preservando marcas de formato.

    Procesa recursivamente los nodos hijos para manejar negritas, itálicas, etc.

    Args:
        node (etree._Element): El nodo padre.

    Returns:
        str: Cadena HTML con el contenido de los hijos.
    """
    segments: List[str] = []
    if node.text:
        segments.append(escape(node.text))
    for child in node:
        segments.append(_inline_element_html(child))
        if child.tail:
            segments.append(escape(child.tail))
    return "".join(segments)

def _inline_element_html(element: etree._Element) -> str:
    """Convierte un elemento XML individual a su representación HTML inline.

    Maneja etiquetas específicas de JATS como xref, italic, bold, sup, sub, ext-link.

    Args:
        element (etree._Element): El elemento a convertir.

    Returns:
        str: Representación HTML del elemento.
    """
    tag = etree.QName(element).localname.lower()
    if tag == "xref" and (element.get("ref-type") or "").lower() == "bibr":
        rid_attr = (element.get("rid") or "").strip()
        first_target = rid_attr.split()[0] if rid_attr else ""
        label_html = _inline_children_html(element) or escape(element.text or "")
        href_attr = f"#{escape(first_target)}" if first_target else "#"
        data_attr = escape(rid_attr)
        return f"<a class=\"citation\" href=\"{href_attr}\" data-rid=\"{data_attr}\">{label_html}</a>"
    if tag in {"italic", "i"}:
        return f"<em>{_inline_children_html(element)}</em>"
    if tag in {"bold", "b", "strong"}:
        return f"<strong>{_inline_children_html(element)}</strong>"
    if tag in {"underline", "u"}:
        return f"<span class=\"underline\">{_inline_children_html(element)}</span>"
    if tag == "sup":
        return f"<sup>{_inline_children_html(element)}</sup>"
    if tag == "sub":
        return f"<sub>{_inline_children_html(element)}</sub>"
    if tag == "ext-link":
        href = element.get(_XLINK_HREF) or element.get("href") or ""
        label = _inline_children_html(element) or escape(element.text or href)
        if href:
            return f"<a class=\"external-link\" href=\"{escape(href)}\" target=\"_blank\"
rel=\"noopener\">{label}</a>"
```

Código Fuente - Transformador XML JATS

```
        return label
    return _inline_children_html(element)

def _xpath(node: etree._Element, expression: str, namespaces: Dict[str, str]) -> List[Any]:
    """Ejecuta una expresión XPath, manejando posibles prefijos de namespace faltantes.

    Intenta usar el namespace 'j' (JATS) y si falla intenta sin prefijo como fallback.

    Args:
        node (etree._Element): Nodo sobre el cual ejecutar XPath.
        expression (str): Expresión XPath.
        namespaces (Dict[str, str]): Mapa de namespaces.

    Returns:
        List[Any]: Lista de resultados encontrados.
    """
    result = node.xpath(expression, namespaces=namespaces)
    if result or "j:" not in expression:
        return result
    fallback_expression = expression.replace("j:", "")
    try:
        return node.xpath(fallback_expression)
    except etree.XPathError:
        return []

def _first_text(node: etree._Element, expression: str, namespaces: Dict[str, str]) -> str:
    """Retorna el texto del primer resultado de una búsqueda XPath.

    Args:
        node (etree._Element): Nodo raíz.
        expression (str): Expresión XPath.
        namespaces (Dict[str, str]): Namespaces.

    Returns:
        str: Texto limpio del primer match, o cadena vacía si no hay coincidencias.
    """
    results = _xpath(node, expression, namespaces)
    if not results:
        return ""
    candidate = results[0]
    if isinstance(candidate, str):
        return _clean_whitespace(candidate)
    if isinstance(candidate, etree._Element):
        return _node_text(candidate)
    return _clean_whitespace(str(candidate))

def _sanitize_xml(raw_xml: str) -> str:
    """Limpia el XML crudo para asegurar que sea parseable.

    Elimina caracteres de control inválidos, ajusta declaraciones XML/DOCTYPE,
    y asegura que exista un nodo raíz <article>.

    Args:
        raw_xml (str): Contenido XML original.

    Returns:
```

Código Fuente - Transformador XML JATS

str: XML sanitizado y seguro para lxml.

Raises:

ValueError: Si no se encuentra un nodo <article> válido.

```
"""
cleaned = raw_xml.replace("\u000b", " ").replace("\u000c", " ")
decl_match = re.search(r"<\?xml[^\>]*\>", cleaned)
decl = decl_match.group(0) if decl_match else '<?xml version="1.0" encoding="UTF-8"?>'
doctype_match = re.search(r"<!DOCTYPE[^\>]*\>", cleaned)
doctype = doctype_match.group(0) if doctype_match else ""
article_match = re.search(r"<article[\s\S]*\>/article>", cleaned)
if not article_match:
    raise ValueError("No se encontró un nodo <article> válido en el archivo JATS.")
article_xml = article_match.group(0)
payload = "\n".join(part for part in [decl, doctype, article_xml] if part)
payload = _SANITIZE_ENTITY_RE.sub("&", payload)
return payload
```

```
def _ensure_namespace(root: etree._Element) -> Dict[str, str]:
```

"""Descubre y normaliza los namespaces del documento XML.

Asigna el prefijo 'j' al namespace por defecto o JATS si no existe.

Args:

root (etree._Element): Nodo raíz del documento.

Returns:

Dict[str, str]: Mapa de prefijos a URIs.

```
"""
namespaces = {prefix: uri for prefix, uri in root.nsmap.items() if prefix}
default_ns = root.nsmap.get(None)
if default_ns:
    namespaces.setdefault("j", default_ns)
namespaces.setdefault("j", "http://www.ncbi.nlm.nih.gov/JATS1")
namespaces.setdefault("xlink", "http://www.w3.org/1999/xlink")
return namespaces
```

```
def parse_jats(xml_path: str) -> Dict[str, Any]:
```

"""Parsea un archivo JATS XML y extrae metadatos y contenido estructurado.

Extrae título, autores, secciones, figuras, tablas y referencias.

Args:

xml_path (str): Ruta al archivo XML.

Returns:

Dict[str, Any]: Diccionario con toda la información extraída del artículo.

```
"""
raw_xml = Path(xml_path).read_text(encoding="utf-8")
sanitized = _sanitize_xml(raw_xml)
parser = etree.XMLParser(
    resolve_entities=False,
    remove_blank_text=True,
    load_dtd=False,
    no_network=True,
    dtd_validation=False,
)
root = etree.fromstring(sanitized.encode("utf-8"), parser)
```

Código Fuente - Transformador XML JATS

```
namespaces = _ensure_namespace(root)

front = root.find("./{*}front")
body = root.find("./{*}body")
back = root.find("./{*}back")
article_meta = front.find("./{*}article-meta") if front is not None else None

data: Dict[str, Any] = {
    "title": "",
    "translated_title": "",
    "journal_title": "",
    "publisher": "",
    "pub_date": "",
    "volume": "",
    "issue": "",
    "fpage": "",
    "lpage": "",
    "elocation": "",
    "doi": "",
    "authors": [],
    "keywords": [],
    "abstract_paragraphs": [],
    "sections": [],
    "figures": [],
    "tables": [],
    "references": [],
}

if article_meta is not None:
    data["title"] = _first_text(article_meta, ".//j:article-title", namespaces)
    data["translated_title"] = _first_text(article_meta, ".//j:translated-title", namespaces)
    data["journal_title"] = _first_text(article_meta, ".//j:journal-title", namespaces)
    data["publisher"] = _first_text(article_meta, ".//j:publisher-name", namespaces)
    data["pub_date"] = _extract_pub_date(article_meta, namespaces)
    data["volume"] = _first_text(article_meta, ".//j:volume", namespaces)
    data["issue"] = _first_text(article_meta, ".//j:issue", namespaces)
    data["fpage"] = _first_text(article_meta, ".//j:fpage", namespaces)
    data["lpage"] = _first_text(article_meta, ".//j:lpage", namespaces)
    data["elocation"] = _first_text(article_meta, ".//j:elocation-id", namespaces)
    data["doi"] = _first_text(article_meta, ".//j:article-id[@pub-id-type='doi']", namespaces)
    data["abstract_paragraphs"] = _extract_paragraphs(article_meta, ".//j:abstract/j:p", namespaces)
    data["keywords"] = _extract_keywords(article_meta, namespaces)
    data["authors"] = _extract_authors(article_meta, namespaces)

if body is not None:
    data["sections"] = _extract_sections(body, namespaces)
    data["figures"] = _extract_figures(body, namespaces)
    data["tables"] = _extract_tables(body, namespaces)

if back is not None:
    data["references"] = _extract_references(back, namespaces)

return data

def _extract_pub_date(article_meta: etree._Element, namespaces: Dict[str, str]) -> str:
    """Extrae la fecha de publicación en formato DD/MM/YYYY o parcial.

    Args:
        article_meta (etree._Element): Elemento article-meta.
        namespaces (Dict[str, str]): Mapa de namespaces.
```


Código Fuente - Transformador XML JATS

Returns:

```
    str: Fecha formateada o cadena vacía.
"""
for node in _xpath(article_meta, ".//j:pub-date", namespaces):
    day = _first_text(node, ".//j:day", namespaces)
    month = _first_text(node, ".//j:month", namespaces)
    year = _first_text(node, ".//j:year", namespaces)
    if year and month and day:
        return f"{day}/{month}/{year}"
    if year and month:
        return f"{month}/{year}"
    if year:
        return year
return ""
```

```
def _extract_keywords(article_meta: etree._Element, namespaces: Dict[str, str]) -> List[str]:
    """Extrae las palabras clave del artículo.
```

Args:

```
    article_meta (etree._Element): Elemento article-meta.
    namespaces (Dict[str, str]): Namespaces.
```

Returns:

```
    List[str]: Lista de palabras clave.
"""
keywords: List[str] = []
for kwd in _xpath(article_meta, ".//j:kwd-group/j:kwd", namespaces):
    text = _node_text(kwd)
    if text:
        keywords.append(text)
return keywords
```

```
def _extract_authors(article_meta: etree._Element, namespaces: Dict[str, str]) -> List[Dict[str, Any]]:
    """Extrae la lista completa de autores y sus metadatos.
```

Procesa nombres, afiliaciones, correspondencia y correos electrónicos.

Args:

```
    article_meta (etree._Element): Elemento article-meta.
    namespaces (Dict[str, str]): Namespaces.
```

Returns:

```
    List[Dict[str, Any]]: Lista de diccionarios con info de cada autor.
"""
affiliations: Dict[str, str] = {}
for aff in _xpath(article_meta, ".//j:aff", namespaces):
    aff_id = aff.get(_XML_ID) or aff.get("id")
    if aff_id:
        affiliations[aff_id] = _node_text(aff)

corresp_notes: Dict[str, str] = {}
for cor in _xpath(article_meta, ".//j:author-notes/j:corresp", namespaces):
    cor_id = cor.get(_XML_ID) or cor.get("id")
    email_value = _first_text(cor, ".//j:email", namespaces) or _node_text(cor)
    if cor_id and email_value:
        corresp_notes[cor_id] = email_value
```

Código Fuente - Transformador XML JATS

```
authors: List[Dict[str, Any]] = []
for contrib in _xpath(article_meta, ".//j:contrib-group/j:contrib[@contrib-type='author']", namespaces):
    surname = _first_text(contrib, ".//j:name/j:surname", namespaces)
    given = _first_text(contrib, ".//j:name/j:given-names", namespaces)
    collective = _first_text(contrib, ".//j:collab", namespaces)
    full_name = collective or " ".join(part for part in [given, surname] if part)
    if not full_name:
        continue

    is_corresponding = contrib.get("corresp", "").lower() == "yes"
    corresp_rids = [xref.get("rid") for xref in _xpath(contrib, ".//j:xref[@ref-type='corresp']",
namespaces) if xref.get("rid")]

    author_affs: List[str] = []
    for xref in _xpath(contrib, ".//j:xref[@ref-type='aff']", namespaces):
        rid = xref.get("rid")
        if rid and rid in affiliations:
            author_affs.append(affiliations[rid])
    if not author_affs:
        inline_aff = contrib.find(".//{*}aff")
        inline_text = _node_text(inline_aff) if inline_aff is not None else ""
        if inline_text:
            author_affs.append(inline_text)

    emails: List[str] = []
    for email in _xpath(contrib, ".//j:email", namespaces):
        value = _node_text(email)
        if value:
            emails.append(value)

    if corresp_rids:
        for cor_id in corresp_rids:
            note_email = corresp_notes.get(cor_id)
            if note_email and note_email not in emails:
                emails.append(note_email)

    orcid_value = ""
    for contrib_id in _xpath(contrib, ".//j:contrib-id[@contrib-id-type='orcid']", namespaces):
        raw = _node_text(contrib_id)
        if raw:
            orcid_value = raw.strip()
            break

    if not is_corresponding:
        is_corresponding = bool(corresp_rids or contrib.get("corresp"))

    authors.append(
        {
            "full": full_name,
            "affiliations": author_affs,
            "emails": emails,
            "orcid": orcid_value,
            "corresponding": is_corresponding,
        }
    )
return authors
```

```
def _extract_paragraphs(parent: etree.Element, expression: str, namespaces: Dict[str, str]) -> List[str]:
    """Extrae párrafos de texto como HTML inline dado un XPath.
```

Código Fuente - Transformador XML JATS

```
Args:
    parent (etree._Element): Elemento padre.
    expression (str): XPath para encontrar los párrafos.
    namespaces (Dict[str, str]): Namespaces.

Returns:
    List[str]: Lista de párrafos en HTML.
"""
paragraphs: List[str] = []
for element in _xpath(parent, expression, namespaces):
    html = _inline_children_html(element)
    if html:
        paragraphs.append(html)
return paragraphs

def _extract_sections(body: etree._Element, namespaces: Dict[str, str]) -> List[Dict[str, Any]]:
    """Extrae jerarquía de secciones del cuerpo del artículo recursivamente.

    Args:
        body (etree._Element): Elemento body o section padre.
        namespaces (Dict[str, str]): Namespaces.

    Returns:
        List[Dict[str, Any]]: Lista de secciones con títulos, párrafos y subsecciones.
    """
    def serialize(sec: etree._Element, index: int) -> Dict[str, Any]:
        sec_id = sec.get(_XML_ID) or sec.get("id") or f"sec-{index}"
        children = _xpath(sec, "./j:sec", namespaces)
        return {
            "id": sec_id,
            "label": _first_text(sec, "./j:label", namespaces),
            "title": _first_text(sec, "./j:title", namespaces) or f"Sección {index}",
            "paragraphs": _extract_paragraphs(sec, "./j:p", namespaces),
            "subsections": [serialize(child, idx) for idx, child in enumerate(children, start=1)],
        }

    sections: List[Dict[str, Any]] = []
    for idx, sec in enumerate(_xpath(body, "./j:sec", namespaces), start=1):
        sections.append(serialize(sec, idx))
    return sections

def _extract_figures(body: etree._Element, namespaces: Dict[str, str]) -> List[Dict[str, Any]]:
    """Extrae figuras del artículo.

    Args:
        body (etree._Element): Cuerpo del artículo.
        namespaces (Dict[str, str]): Namespaces.

    Returns:
        List[Dict[str, Any]]: Lista de figuras con metadatos y URL de imagen.
    """
    figures: List[Dict[str, Any]] = []
    for fig in _xpath(body, "./j:fig", namespaces):
        fig_id = fig.get(_XML_ID) or fig.get("id") or f"fig-{len(figures) + 1}"
        label = _first_text(fig, "./j:label", namespaces)
        caption = _first_text(fig, "./j:caption", namespaces)
        graphic = fig.find("./{*}graphic")
        href = ""
```

Código Fuente - Transformador XML JATS

```
if graphic is not None:
    href = graphic.get(f"{{{namespaces['xlink']}}}href", "")
    figures.append(
        {
            "id": fig_id,
            "label": label,
            "caption": caption,
            "href": href,
        }
    )
return figures
```

```
def _extract_tables(body: etree._Element, namespaces: Dict[str, str]) -> List[Dict[str, Any]]:
    """Extrae tablas del artículo.
```

Args:

body (etree._Element): Cuerpo del artículo.
namespaces (Dict[str, str]): Namespaces.

Returns:

List[Dict[str, Any]]: Lista de tablas con sus filas y celdas.
"""

```
tables: List[Dict[str, Any]] = []
for wrap in _xpath(body, ".//j:table-wrap", namespaces):
    table_id = wrap.get(_XML_ID) or wrap.get("id") or f"table-{len(tables) + 1}"
    label = _first_text(wrap, ".//j:label", namespaces)
    caption = _first_text(wrap, ".//j:caption", namespaces)
    table_node = wrap.find(".//{*}table")
    rows: List[List[str]] = []
    if table_node is not None:
        for row in table_node.findall(".//{*}tr"):
            cells: List[str] = []
            for cell in list(row.findall(".//{*}th")) + list(row.findall(".//{*}td")):
                cells.append(_node_text(cell))
            if cells:
                rows.append(cells)
    tables.append(
        {
            "id": table_id,
            "label": label,
            "caption": caption,
            "rows": rows,
        }
    )
return tables
```

```
def _extract_references(back: etree._Element, namespaces: Dict[str, str]) -> List[Dict[str, str]]:
    """Extrae las referencias bibliográficas.
```

Args:

back (etree._Element): Sección back del artículo.
namespaces (Dict[str, str]): Namespaces.

Returns:

List[Dict[str, str]]: Lista de referencias con ID, etiqueta y HTML renderizado.
"""

```
references: List[Dict[str, str]] = []
for ref in _xpath(back, ".//j:ref-list/j:ref", namespaces):
```

Código Fuente - Transformador XML JATS

```
ref_id = ref.get(_XML_ID) or ref.get("id") or ""
label = _first_text(ref, "./j:label", namespaces)
citation_node = ref.find("./j:mixed-citation", namespaces) or ref.find("./j:element-citation",
namespaces)
    body_html = _inline_children_html(citation_node) if citation_node is not None else
_inline_children_html(ref)
    references.append({"id": ref_id, "label": label, "html": body_html})
return references
```

```
def _collect_section_entries(sections: List[Dict[str, Any]], prefix: str = "", level: int = 1) ->
List[Tuple[int, str, str, str]]:
```

```
    """Aplana la jerarquía de secciones para generar el índice (TOC).
```

```
    Args:
```

```
        sections (List[Dict[str, Any]]): Lista de secciones anidadas.
```

```
        prefix (str, optional): Prefijo de numeración acumulado.
```

```
        level (int, optional): Nivel de profundidad actual.
```

```
    Returns:
```

```
        List[Tuple[int, str, str, str]]: Lista de tuplas (nivel, id, etiqueta, título).
```

```
    """
```

```
    entries: List[Tuple[int, str, str, str]] = []
```

```
    for idx, section in enumerate(sections, start=1):
```

```
        number = f"{prefix}{idx}" if prefix else str(idx)
```

```
        label = section.get("label") or number
```

```
        entries.append((level, section["id"], label, section["title"]))
```

```
        entries.extend(_collect_section_entries(section["subsections"], f"{number}.", level + 1))
```

```
    return entries
```

```
def _render_keywords(keywords: Iterable[str]) -> str:
```

```
    """Genera el HTML para la sección de palabras clave.
```

```
    Args:
```

```
        keywords (Iterable[str]): Lista de palabras clave.
```

```
    Returns:
```

```
        str: HTML renderizado o cadena vacía si no hay keywords.
```

```
    """
```

```
    items = [f"<li>{escape(keyword)}</li>" for keyword in keywords if keyword]
```

```
    if not items:
```

```
        return ""
```

```
    return """
```

```
<section class=\"panel\">
```

```
    <h3>Palabras clave</h3>
```

```
    <ul class=\"keyword-list\">{items}</ul>
```

```
</section>
```

```
""".replace("{items}", "".join(items))
```

```
def _render_authors(authors: List[Dict[str, Any]]) -> str:
```

```
    """Genera el HTML para la lista de autores y sus filiaciones.
```

```
    Args:
```

```
        authors (List[Dict[str, Any]]): Metadatos de autores.
```

```
    Returns:
```

```
        str: HTML renderizado o cadena vacía.
```

Código Fuente - Transformador XML JATS

```
"""
if not authors:
    return ""
author_items: List[str] = []
for author in authors:
    name_classes = ["author-name"]
    if author.get("corresponding"):
        name_classes.append("author-name--corresponding")
    header_parts = [f"<span class='{ ' '.join(name_classes)}\>{escape(author['full'])}</span>"]
    if author.get("corresponding"):
        header_parts.append("<span class='author-corresponding\>Autor de correspondencia</span>")
    sections: List[str] = [f"<div class='author-header\>{' '.join(header_parts)}</div>"]

    if author.get("affiliations"):
        affs = "; ".join(escape(aff) for aff in author["affiliations"] if aff)
        if affs:
            sections.append(f"<div class='author-affiliations\>{affs}</div>")

    contact_links: List[str] = []
    orcid_value = (author.get("orcid") or "").strip()
    if orcid_value:
        orcid_url = orcid_value if orcid_value.lower().startswith("http") else
f"https://orcid.org/{orcid_value}"
        orcid_display = orcid_url.rstrip("/").split("/")[-1] or orcid_url
        contact_links.append(
            f"<a class='author-link author-orcid' href='{escape(orcid_url)}' target='_blank'
rel='noopener\>ORCID: {escape(orcid_display)}</a>"
        )

    for email in author.get("emails", []):
        if email:
            email_classes = ["author-link", "author-email"]
            if author.get("corresponding"):
                email_classes.append("author-email--corresponding")
            contact_links.append(
                f"<a class='{ ' '.join(email_classes)}' href='mailto:{escape(email)}'\>Email:
{escape(email)}</a>"
            )

    if contact_links:
        link_items = "".join(f"<span class='author-link-item\>{link}</span>" for link in contact_links)
        sections.append(f"<div class='author-links\>{link_items}</div>")

    author_items.append(f"<li class='author\>{' '.join(sections)}</li>")

return ""
<section class='panel\>
    <h3>Autores</h3>
    <ul class='author-list\>{items}</ul>
</section>
"".replace("{items}", "".join(author_items))

def _render_paragraphs(paragraphs: Iterable[str]) -> str:
    """Envuelve los párrafos en etiquetas <p>.

    Args:
        paragraphs (Iterable[str]): Lista de párrafos ya escapados/procesados.

    Returns:
        str: HTML concatenado.
```

Código Fuente - Transformador XML JATS

```
"""
return "".join(f"<p>{paragraph}</p>" for paragraph in paragraphs if paragraph)

def _render_sections(sections: List[Dict[str, Any]], depth: int = 1, prefix: str = "") -> str:
    """Renderiza las secciones y subsecciones de forma recursiva.

    Args:
        sections (List[Dict[str, Any]]): Lista de secciones.
        depth (int, optional): Profundidad actual (para jerarquía h2-h6).
        prefix (str, optional): Prefijo de numeración (ej. "1.1").

    Returns:
        str: HTML de todas las secciones renderizadas.
    """
    html_parts: List[str] = []
    heading_tags = {1: "h2", 2: "h3", 3: "h4", 4: "h5"}
    for idx, section in enumerate(sections, start=1):
        number = f"{prefix}{idx}" if prefix else str(idx)
        heading_tag = heading_tags.get(depth, "h6")
        title = escape(section["title"])
        label_text = section.get("label") or number
        label = escape(label_text) if label_text else ""
        section_id = escape(section["id"])
        classes = ["article-section", f"level-{depth}"]
        if depth == 1:
            classes.extend(["main-section", "surface"])
        else:
            classes.append("subsection")
        html_parts.append(f"<section id='{section_id}' class='{ ' '.join(classes)} '>")
        heading_parts = ["<header class='section-header'>"]
        number_text = label
        if number_text and not number_text.endswith('.'):
            number_text = f"{number_text}."
        number_html = f"<span class='section-number'>{number_text}</span>" if number_text else ""
        heading_parts.append(
            f"<{heading_tag} class='section-heading'>{number_html}<span
class='section-title'>{title}</span></{heading_tag}>"
        )
        heading_parts.append("</header>")
        html_parts.append("".join(heading_parts))
        body_html = _render_paragraphs(section["paragraphs"])
        if body_html:
            html_parts.append(f"<div class='section-body'>{body_html}</div>")
        if section["subsections"]:
            html_parts.append("<div class='subsection-group'>")
            html_parts.append(_render_sections(section["subsections"], depth + 1, f"{number}."))
            html_parts.append("</div>")
        html_parts.append("</section>")
    return "".join(html_parts)

def _render_figures(figures: List[Dict[str, Any]]) -> str:
    """Genera HTML para las figuras.

    Args:
        figures (List[Dict[str, Any]]): Lista de figuras.

    Returns:
        str: HTML renderizado.
```

Código Fuente - Transformador XML JATS

```
"""
if not figures:
    return ""
figure_blocks: List[str] = []
for fig in figures:
    caption = escape(fig.get("caption") or "")
    label = escape(fig.get("label") or "Figura")
    href = escape(fig.get("href") or "")
    alt = caption or label
    media = f"<img src='{href}' alt='{alt}' loading='lazy'>" if href else ""
    figure_blocks.append(
        """
        <figure id="{id}" class="media">
            {media}
            <figcaption><span class="media-label">{label}</span> {caption}</figcaption>
        </figure>
        """.replace("{id}", escape(fig.get("id") or ""))
        .replace("{media}", media)
        .replace("{label}", label)
        .replace("{caption}", caption)
    )
return """
<section class="panel" id="figures">
    <h3>Figuras</h3>
    {figures}
</section>
""".replace("{figures}", "".join(figure_blocks))

def _render_tables(tables: List[Dict[str, Any]]) -> str:
    """Genera HTML para las tablas.

    Args:
        tables (List[Dict[str, Any]]): Lista de tablas.

    Returns:
        str: HTML renderizado.
    """
    if not tables:
        return ""
    table_blocks: List[str] = []
    for table in tables:
        rows_html: List[str] = []
        for row in table["rows"]:
            cells_html = "".join(f"<td>{escape(cell)}</td>" for cell in row)
            rows_html.append(f"<tr>{cells_html}</tr>")
        label = escape(table.get("label") or "Tabla")
        caption = escape(table.get("caption") or "")
        table_blocks.append(
            """
            <figure id="{id}" class="table-wrap">
                <figcaption><span class="media-label">{label}</span> {caption}</figcaption>
                <div class="table-scroll">
                    <table>{rows}</table>
                </div>
            </figure>
            """.replace("{id}", escape(table.get("id") or ""))
            .replace("{label}", label)
            .replace("{caption}", caption)
            .replace("{rows}", "".join(rows_html))
        )
    )
```


Código Fuente - Transformador XML JATS

```
return ""
<section class=\"panel\" id=\"tables\">
  <h3>Tablas</h3>
  {tables}
</section>
"".replace("{tables}", "".join(table_blocks))

def _render_references(references: List[Dict[str, str]]) -> str:
    """Genera HTML para la lista de referencias.

    Args:
        references (List[Dict[str, str]]): Lista de referencias.

    Returns:
        str: HTML renderizado.
    """
    if not references:
        return ""
    items: List[str] = []
    for ref in references:
        label = escape(ref.get("label") or "")
        body = ref.get("html") or ""
        item_id = escape(ref.get("id") or "")
        id_attr = f" id=\"{item_id}\"" if item_id else ""
        if label:
            items.append(f"<li{id_attr}><span class=\"reference-label\">{label}</span> {body}</li>")
        else:
            items.append(f"<li{id_attr}>{body}</li>")
    return ""
<section class=\"panel\" id=\"references\">
  <h3>Referencias</h3>
  <ol class=\"reference-list\">{items}</ol>
</section>
"".replace("{items}", "".join(items))

def _get_logo_base64() -> str:
    """Lee el archivo de logo y lo convierte a Base64."""
    try:
        logo_path = Path("resources/logo.png")
        if logo_path.exists():
            import base64
            encoded = base64.b64encode(logo_path.read_bytes()).decode("utf-8")
            return f"data:image/png;base64,{encoded}"
    except Exception:
        pass
    return ""

def _render_sidebar(doc: Dict[str, Any]) -> str:
    """Genera el panel lateral con la tabla de contenidos (TOC) y metadatos.

    Args:
        doc (Dict[str, Any]): Datos completos del artículo.

    Returns:
        str: HTML del sidebar.
    """
    sections = doc.get("sections", [])
    entries = _collect_section_entries(sections)
```

Código Fuente - Transformador XML JATS

```
nav_items: List[str] = []
for level, sec_id, label, title in entries:
    nav_items.append(
        """
        <li class=\\"toc-item level-{"level}"\">
            <a href=\\"#{href}"\">
                <span class=\\"toc-marker\\" aria-hidden=\\"true\\"></span>
                <span class=\\"toc-number\\">{label}</span>
                <span class=\\"toc-title\\">{title}</span>
            </a>
        </li>
        """
        .replace("{level}", str(level))
        .replace("{href}", escape(sec_id))
        .replace("{label}", escape(label))
        .replace("{title}", escape(title))
    )
toc_html = (
    "".join(nav_items)
    if nav_items
    else "<li class=\\"toc-item\\"><span class=\\"toc-title\\">Contenido no disponible</span></li>"
)

meta_rows: List[str] = []
journal = doc.get("journal_title")
if journal:
    meta_rows.append(f"<li><span>Revista:</span> {escape(journal)}</li>")
publisher = doc.get("publisher")
if publisher:
    meta_rows.append(f"<li><span>Editorial:</span> {escape(publisher)}</li>")
issue_bits = [doc.get("volume"), doc.get("issue")]
issue = "".join(bit for bit in issue_bits if bit)
if issue:
    meta_rows.append(f"<li><span>Volumen/Edición:</span> {escape(issue)}</li>")
pages = "".join(filter(None, [doc.get("fpage"), doc.get("lpage")]))
if pages:
    meta_rows.append(f"<li><span>Páginas:</span> {escape(pages)}</li>")
elocation = doc.get("elocation")
if elocation:
    meta_rows.append(f"<li><span>Ubicación electrónica:</span> {escape(elocation)}</li>")
doi = doc.get("doi")
if doi:
    meta_rows.append(f"<li><span>DOI:</span> <a href=\\"https://doi.org/{escape(doi)}\\">{escape(doi)}</a></li>")
    pub_date = doc.get("pub_date")
    if pub_date:
        meta_rows.append(f"<li><span>Fecha:</span> {escape(pub_date)}</li>")

logo_src = _get_logo_base64()
logo_html = ""
if logo_src:
    logo_html = f"""
    <div class=\\"logo-container\\" style=\\"text-align: center; margin-bottom: 20px;\">
        <img src=\\"{logo_src}" alt=\\"Logo Revista\\" style=\\"max-width: 80%; height: auto;\">
    </div>
    """

template = """
<aside class=\\"sidebar\\" id=\\"sidebar\\" aria-label=\\"Índice de contenido\\" aria-hidden=\\"false\\">
    <div class=\\"sidebar-header\\">
        {logo}
    <div class=\\"sidebar-title-row\\">
```

Código Fuente - Transformador XML JATS

```
        <h2>Contenido</h2>
        <button class=\"sidebar-close\" id=\"sidebarClose\" aria-label=\"Cerrar índice\">?</button>
    </div>
</div>
<nav class=\"toc\">
    <ul>{toc}</ul>
</nav>
<section class=\"panel meta\">
    <h3>Ficha técnica</h3>
    <ul class=\"meta-list\">{meta}</ul>
</section>
    {keywords}
</aside>
"""

return (
    template.replace("{toc}", toc_html)
    .replace("{meta}", "".join(meta_rows) if meta_rows else "<li>Información no disponible</li>")
    .replace("{keywords}", _render_keywords(doc.get("keywords", [])))
    .replace("{logo}", logo_html)
)

def build_html(doc: Dict[str, Any]) -> str:
    """Construye el documento HTML completo a partir de los datos extraídos.

    Ensambla CSS, JS, metadatos y secciones en una plantilla HTML responsiva.

    Args:
        doc (Dict[str, Any]): Datos estructurados del artículo.

    Returns:
        str: Código fuente HTML completo.
    """
    title = escape(doc.get("title") or "Artículo sin título")
    translated_title = escape(doc.get("translated_title") or "")

    css = """
:root {
  color-scheme: light;
  --font-sans: "Inter", "Segoe UI", -apple-system, BlinkMacSystemFont, sans-serif;
  --bg-body: #f3f4f6;
  --bg-card: #ffffff;
  --accent: #1f3d7a;
  --accent-secondary: #2f4f8f;
  --accent-soft: rgba(31, 61, 122, 0.12);
  --accent-soft-alt: rgba(31, 61, 122, 0.08);
  --border-soft: rgba(15, 23, 42, 0.14);
  --text-main: #1f2937;
  --text-muted: #374151;
  --text-subtle: #4b5563;
  --shadow-soft: 0 24px 48px -32px rgba(15, 23, 42, 0.35);
  --radius-md: 18px;
  --focus-ring: 0 0 0 3px rgba(31, 61, 122, 0.35);
  --bg-soft: rgba(15, 23, 42, 0.04);
  --header-bg: linear-gradient(135deg, #f9fafc, #eef1f7);
}
body[data-theme="dark"] {
  color-scheme: dark;
  --bg-body: #0b1220;
  --bg-card: #121a2f;
```

Código Fuente - Transformador XML JATS

```
--accent: #7aa2ff;
--accent-secondary: #90b4ff;
--accent-soft: rgba(122, 162, 255, 0.22);
--accent-soft-alt: rgba(122, 162, 255, 0.12);
--border-soft: rgba(148, 163, 184, 0.35);
--text-main: #e2e8f0;
--text-muted: #cbd5f5;
--text-subtle: #a5b4d1;
--shadow-soft: 0 28px 56px -32px rgba(0, 0, 0, 0.65);
--focus-ring: 0 0 0 3px rgba(122, 162, 255, 0.5);
--bg-soft: rgba(122, 162, 255, 0.12);
--header-bg: linear-gradient(135deg, #10182b, #131f36);
}
body[data-contrast="high"] {
  --bg-body: #ffffff;
  --bg-card: #ffffff;
  --accent: #0b1120;
  --accent-secondary: #1f2937;
  --accent-soft: rgba(11, 17, 32, 0.12);
  --accent-soft-alt: rgba(11, 17, 32, 0.08);
  --border-soft: rgba(11, 17, 32, 0.35);
  --text-main: #0b1120;
  --text-muted: #111827;
  --text-subtle: #1f2937;
  --shadow-soft: none;
  --focus-ring: 0 0 0 3px rgba(11, 17, 32, 0.4);
  --bg-soft: rgba(11, 17, 32, 0.08);
  --header-bg: linear-gradient(135deg, #ffffff, #e5e7eb);
}
body[data-theme="dark"][data-contrast="high"] {
  --bg-body: #020817;
  --bg-card: #061226;
  --accent: #f3c969;
  --accent-secondary: #f0d893;
  --accent-soft: rgba(243, 201, 105, 0.28);
  --accent-soft-alt: rgba(243, 201, 105, 0.18);
  --border-soft: rgba(243, 201, 105, 0.5);
  --text-main: #f8fafc;
  --text-muted: #e2e8f0;
  --text-subtle: #cbd5f5;
  --focus-ring: 0 0 0 3px rgba(243, 201, 105, 0.55);
  --bg-soft: rgba(243, 201, 105, 0.18);
  --header-bg: linear-gradient(135deg, #0b172c, #152441);
}
*, *::before, *::after {
  box-sizing: border-box;
}
html {
  scroll-behavior: smooth;
}
body {
  margin: 0;
  font-family: var(--font-sans);
  background: var(--bg-body);
  color: var(--text-main);
  transition: background 0.25s ease, color 0.25s ease;
  overflow-x: hidden;
}
body.sidebar-lock {
  overflow: hidden;
}
a {
```

Código Fuente - Transformador XML JATS

```
    color: var(--accent);
    text-decoration: none;
}
a:hover, a:focus {
    text-decoration: underline;
}
a:focus-visible, button:focus-visible {
    outline: none;
    box-shadow: var(--focus-ring);
}
button {
    font: inherit;
}
.app {
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}
.app-header {
    padding: clamp(20px, 4vw, 36px) clamp(16px, 5vw, 48px) 12px;
    background: var(--bg-body);
    margin-bottom: clamp(16px, 4vw, 32px);
}
.header-panel {
    margin: 0 auto;
    width: 100%;
    max-width: 960px;
    background: var(--bg-card);
    border: 1px solid var(--border-soft);
    border-radius: var(--radius-md);
    box-shadow: var(--shadow-soft);
    padding: clamp(24px, 3vw, 40px);
    display: flex;
    flex-direction: column;
    gap: clamp(16px, 2.5vw, 24px);
}
.branding {
    flex: 1 1 auto;
    width: 100%;
    position: relative;
    z-index: 1;
}
.branding h1 {
    font-size: clamp(2rem, 2.8vw, 2.75rem);
    margin: 0;
    letter-spacing: -0.02em;
    color: var(--text-main);
}
.branding .translated-title {
    margin: 8px 0 0;
    font-size: clamp(1.1rem, 2vw, 1.35rem);
    color: var(--text-subtle);
}
.control-bar {
    display: flex;
    align-items: center;
    gap: 12px;
    flex-wrap: wrap;
    justify-content: flex-start;
    width: 100%;
    border-top: 1px solid var(--border-soft);
    padding-top: clamp(12px, 2vw, 18px);
}
```

Código Fuente - Transformador XML JATS

```
}
.control-bar button {
  background: transparent;
  border: 1px solid var(--border-soft);
  color: var(--text-main);
  padding: 10px 16px;
  border-radius: 999px;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  transition: transform 0.2s ease, box-shadow 0.2s ease, background 0.2s ease;
}
.control-bar button:hover {
  transform: translateY(-1px);
  background: var(--accent-soft);
  box-shadow: var(--shadow-soft);
}
.control-bar button .state {
  font-weight: 500;
}
.layout {
  position: relative;
  padding: 0 clamp(16px, 4vw, 56px) 48px;
  flex: 1 1 auto;
}
.sidebar {
  width: min(340px, 82vw);
  background: var(--bg-card);
  border: 1px solid var(--border-soft);
  border-radius: 0 24px 24px 0;
  box-shadow: var(--shadow-soft);
  padding: 28px 24px;
  position: fixed;
  top: 0;
  left: 0;
  height: 100vh;
  overflow: hidden auto;
  transform: translateX(-110%);
  transition: transform 0.32s ease;
  backdrop-filter: blur(22px);
  z-index: 22;
}
body.sidebar-open .sidebar {
  transform: translateX(0);
}
.sidebar-header {
  margin-bottom: 20px;
}
.sidebar-title-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.sidebar-header h2 {
  margin: 0;
  font-size: 1.3rem;
  color: var(--accent);
}
.sidebar-close {
  background: transparent;
  border: none;
```

Código Fuente - Transformador XML JATS

```
color: var(--text-subtle);
font-size: 1.2rem;
cursor: pointer;
display: inline-flex;
align-items: center;
justify-content: center;
}
.sidebar .panel {
margin-top: 24px;
}
.sidebar .panel h3 {
margin: 0 0 12px;
font-size: 1rem;
text-transform: uppercase;
letter-spacing: 0.06em;
color: var(--text-subtle);
}
.toc ul, .meta-list, .keyword-list {
list-style: none;
padding: 0;
margin: 0;
display: grid;
gap: 10px;
}
.toc-item a {
display: grid;
grid-template-columns: auto 1fr;
align-items: center;
gap: 12px;
padding: 10px 14px;
border-radius: 12px;
color: var(--text-main);
background: transparent;
transition: background 0.2s ease, transform 0.2s ease;
line-height: 1.35;
font-weight: 500;
}
.toc-item a:hover {
background: var(--accent-soft);
transform: translateX(4px);
}
.toc-marker {
display: none;
}
.toc-number {
font-weight: 600;
color: var(--text-main);
min-width: 3ch;
font-size: 1rem;
font-variant-numeric: lining-nums;
}
.toc-number:empty {
display: none;
}
.toc-title {
color: var(--text-main);
}
.toc-item.level-2 {
margin-left: 12px;
}
.toc-item.level-3 {
margin-left: 24px;
}
```

Código Fuente - Transformador XML JATS

```
}
.toc-item.level-4 {
  margin-left: 36px;
}
.toc-item.level-2 a {
  font-size: 0.95rem;
}
.toc-item.level-3 a {
  font-size: 0.9rem;
}
.toc-item.level-4 a {
  font-size: 0.85rem;
}
.meta-list span {
  font-weight: 600;
  display: block;
  color: var(--text-subtle);
}
.keyword-list {
  flex-wrap: wrap;
  grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
}
.keyword-list li {
  background: var(--bg-soft);
  color: var(--text-main);
  border: 1px solid var(--border-soft);
  border-radius: 12px;
  padding: 8px 12px;
  font-size: 0.9rem;
}
.main {
  background: transparent;
  width: 100%;
  max-width: 960px;
  margin: 0 auto;
}
.surface {
  background: var(--bg-card);
  border: 1px solid var(--border-soft);
  border-radius: var(--radius-md);
  box-shadow: var(--shadow-soft);
  padding: clamp(24px, 3vw, 48px);
  margin-bottom: 32px;
}
.surface h2, .surface h3, .surface h4, .surface h5, .surface h6 {
  margin-top: 48px;
  margin-bottom: 16px;
  scroll-margin-top: 120px;
}
.article-body {
  margin-bottom: 32px;
}
.section-grid {
  display: grid;
  gap: clamp(18px, 2.8vw, 32px);
  grid-template-columns: minmax(0, 1fr);
}
.article-section {
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}
.article-section.main-section {
  border-left: 6px solid var(--accent);
}
```


Código Fuente - Transformador XML JATS

```
padding-left: clamp(24px, 3vw, 36px);
}
.article-section.main-section:hover {
  box-shadow: 0 24px 48px -36px rgba(31, 61, 122, 0.35);
}
.section-header {
  display: flex;
  align-items: baseline;
}
.section-heading {
  display: inline-flex;
  align-items: baseline;
  gap: 12px;
  margin: 0;
  font-weight: 600;
  letter-spacing: -0.01em;
  color: var(--text-main);
}
.section-number {
  font-weight: 700;
  font-size: 1.1rem;
  color: var(--accent);
  font-variant-numeric: lining-nums;
  line-height: 1;
}
.section-title {
  font-weight: 600;
}
.section-body {
  display: grid;
  gap: 16px;
  margin-top: 12px;
}
.section-body p {
  line-height: 1.75;
  font-size: 1.03rem;
  color: var(--text-muted);
}
.subsection-group {
  display: grid;
  gap: 16px;
  margin-top: 20px;
  padding-left: clamp(14px, 1.8vw, 22px);
  border-left: 3px solid var(--accent-soft);
}
.article-section.subsection {
  padding: 0;
  border: none;
  background: transparent;
  box-shadow: none;
}
.article-section.subsection .section-header h3,
.article-section.subsection .section-header h4,
.article-section.subsection .section-header h5,
.article-section.subsection .section-header h6 {
  color: var(--text-main);
}
.article-section.subsection .section-number {
  color: var(--text-subtle);
  font-size: 0.95rem;
}
.article-section.subsection .section-body p {
```

Código Fuente - Transformador XML JATS

```
    font-size: 1rem;
}
.article-section.empty-state {
    display: flex;
    align-items: center;
    justify-content: center;
    min-height: 180px;
}
.article-section.empty-state p {
    margin: 0;
    color: var(--text-subtle);
    text-align: center;
}
.meta-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 16px;
    margin-top: 16px;
}
.meta-grid .item {
    padding: 16px;
    background: var(--bg-soft);
    border-radius: 12px;
}
.meta-grid .label {
    display: block;
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.08em;
    color: var(--text-subtle);
    margin-bottom: 6px;
}
.author-list {
    list-style: none;
    margin: 0;
    padding: 0;
    display: grid;
    gap: 16px;
}
.author {
    padding: 18px;
    border: 1px solid var(--border-soft);
    border-radius: 14px;
    background: var(--bg-soft);
}
.author-header {
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    gap: 10px;
    margin-bottom: 8px;
}
.author-name {
    font-weight: 600;
    color: var(--text-main);
}
.author-name--corresponding {
    color: var(--accent);
}
.author-corresponding {
    font-size: 0.85rem;
    font-weight: 600;
```

Código Fuente - Transformador XML JATS

```
    color: var(--accent-secondary);
    background: var(--accent-soft);
    border-radius: 999px;
    padding: 2px 10px;
}
.author-affiliations {
    color: var(--text-subtle);
    font-size: 0.95rem;
    margin-bottom: 6px;
}
.author-links {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
    margin-top: 4px;
}
.author-link-item {
    display: inline-flex;
}
.author-link {
    display: inline-flex;
    align-items: center;
    gap: 6px;
    color: var(--accent-secondary);
    font-weight: 500;
}
.author-link:hover {
    text-decoration: underline;
}
.author-email--corresponding {
    font-weight: 600;
    color: var(--accent);
}
.abstract p {
    font-size: 1.05rem;
    line-height: 1.75;
    color: var(--text-muted);
}
.panel {
    margin-bottom: 32px;
    padding: 24px;
    background: var(--bg-card);
    border: 1px solid var(--border-soft);
    border-radius: var(--radius-md);
    box-shadow: var(--shadow-soft);
}
.media {
    margin: 0 0 24px;
    background: var(--bg-soft);
    padding: 16px;
    border-radius: 12px;
}
.media img {
    max-width: 100%;
    display: block;
    border-radius: 12px;
}
.media-label {
    font-weight: 600;
    color: var(--text-subtle);
    margin-right: 8px;
}
```

Código Fuente - Transformador XML JATS

```
.table-wrap {
  margin: 0 0 24px;
  background: var(--bg-soft);
  padding: 16px;
  border-radius: 12px;
}
.table-scroll {
  overflow-x: auto;
  border-radius: 12px;
}
.table-scroll table {
  width: 100%;
  border-collapse: collapse;
  min-width: 480px;
}
.table-scroll th, .table-scroll td {
  padding: 12px 16px;
  border: 1px solid var(--border-soft);
  text-align: left;
}
.reference-list {
  list-style: decimal;
  margin: 0;
  padding-left: 20px;
  display: grid;
  gap: 12px;
}
.reference-label {
  font-weight: 600;
  color: var(--text-subtle);
  margin-right: 8px;
}
.sidebar-backdrop {
  position: fixed;
  inset: 0;
  background: rgba(15, 23, 42, 0.35);
  backdrop-filter: blur(2px);
  opacity: 0;
  visibility: hidden;
  transition: opacity 0.3s ease;
  z-index: 18;
}
.sidebar-backdrop.visible {
  opacity: 1;
  visibility: visible;
}
.floating-button {
  position: fixed;
  right: clamp(16px, 4vw, 40px);
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 12px 16px;
  background: var(--accent);
  color: #ffffff;
  border: none;
  border-radius: 999px;
  box-shadow: var(--shadow-soft);
  cursor: pointer;
  opacity: 0;
  pointer-events: none;
  transform: translateY(12px);
```

Código Fuente - Transformador XML JATS

```
    transition: opacity 0.25s ease, transform 0.25s ease;
    z-index: 24;
}
.floating-button .icon {
    font-weight: 700;
    font-size: 1rem;
}
.floating-button.visible {
    opacity: 1;
    pointer-events: auto;
    transform: translateY(0);
}
.floating-button:focus-visible {
    outline: none;
    box-shadow: var(--focus-ring);
}
.floating-top {
    bottom: 108px;
}
.floating-menu {
    bottom: 44px;
}
body[data-theme="dark"] .floating-button {
    color: #0b1220;
}
@media (max-width: 720px) {
    .app-header {
        padding-inline: 16px;
    }
    .layout {
        padding-inline: 16px;
    }
    .surface {
        padding: 20px;
    }
    .control-bar {
        flex-wrap: wrap;
    }
}
"""

js = """
(function () {
    const body = document.body;
    const themeToggle = document.getElementById('themeToggle');
    const contrastToggle = document.getElementById('contrastToggle');
    const sidebarToggle = document.getElementById('sidebarToggle');
    const sidebar = document.getElementById('sidebar');
    const sidebarClose = document.getElementById('sidebarClose');
    const sidebarBackdrop = document.getElementById('sidebarBackdrop');
    const scrollTopButton = document.getElementById('scrollTopButton');
    const floatingMenuButton = document.getElementById('floatingMenuButton');
    const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
    const storedTheme = localStorage.getItem('jats-theme');
    const storedContrast = localStorage.getItem('jats-contrast');

    function applyTheme(theme) {
        body.setAttribute('data-theme', theme);
        localStorage.setItem('jats-theme', theme);
        if (themeToggle) {
            themeToggle.querySelector('span.state').textContent = theme === 'dark' ? 'Modo claro' : 'Modo oscuro';
        }
    }
})

```

Código Fuente - Transformador XML JATS

```
}

function applyContrast(contrast) {
  body.setAttribute('data-contrast', contrast);
  localStorage.setItem('jats-contrast', contrast);
  if (contrastToggle) {
    contrastToggle.querySelector('span.state').textContent = contrast === 'high' ? 'Contraste normal' : 'Alto
contraste';
  }
}

applyTheme(storedTheme || (prefersDark ? 'dark' : 'light'));
applyContrast(storedContrast || 'normal');

if (themeToggle) {
  themeToggle.addEventListener('click', () => {
    const current = body.getAttribute('data-theme') === 'dark' ? 'dark' : 'light';
    applyTheme(current === 'dark' ? 'light' : 'dark');
  });
}

if (contrastToggle) {
  contrastToggle.addEventListener('click', () => {
    const current = body.getAttribute('data-contrast') === 'high' ? 'high' : 'normal';
    applyContrast(current === 'high' ? 'normal' : 'high');
  });
}

function updateSidebarAria(isOpen) {
  sidebarToggle?.setAttribute('aria-expanded', isOpen ? 'true' : 'false');
  sidebar?.setAttribute('aria-hidden', isOpen ? 'false' : 'true');
  floatingMenuButton?.setAttribute('aria-expanded', isOpen ? 'true' : 'false');
}

function openSidebar() {
  if (!sidebar) {
    return;
  }
  body.classList.add('sidebar-open', 'sidebar-lock');
  sidebarBackdrop?.classList.add('visible');
  updateSidebarAria(true);
}

function closeSidebar() {
  if (!sidebar) {
    return;
  }
  body.classList.remove('sidebar-open');
  setTimeout(() => body.classList.remove('sidebar-lock'), 250);
  sidebarBackdrop?.classList.remove('visible');
  updateSidebarAria(false);
}

updateSidebarAria(false);

if (sidebarToggle) {
  sidebarToggle.addEventListener('click', () => {
    if (body.classList.contains('sidebar-open')) {
      closeSidebar();
    } else {
      openSidebar();
    }
  })
}
```

Código Fuente - Transformador XML JATS

```
    });
  }

  sidebarClose?.addEventListener('click', closeSidebar);
  sidebarBackdrop?.addEventListener('click', closeSidebar);

  window.addEventListener('keyup', (event) => {
    if (event.key === 'Escape') {
      closeSidebar();
    }
  });

  const tocLinks = sidebar?.querySelectorAll('.toc a') || [];
  tocLinks.forEach((link) => {
    link.addEventListener('click', () => {
      closeSidebar();
    });
  });

  scrollTopButton?.addEventListener('click', () => {
    window.scrollTo({ top: 0, behavior: 'smooth' });
  });

  if (floatingMenuButton) {
    floatingMenuButton.addEventListener('click', () => {
      if (body.classList.contains('sidebar-open')) {
        closeSidebar();
      } else {
        openSidebar();
      }
    });
  }

  function handleScroll() {
    const shouldShow = window.scrollY > 240;
    if (scrollTopButton) {
      scrollTopButton.classList.toggle('visible', shouldShow);
    }
    if (floatingMenuButton) {
      floatingMenuButton.classList.toggle('visible', shouldShow);
    }
  }

  window.addEventListener('scroll', handleScroll, { passive: true });
  handleScroll();
})();
"""

authors_html = _render_authors(doc.get("authors", []))
abstract_html = _render_paragraphs(doc.get("abstract_paragraphs", []))
sections_html = _render_sections(doc.get("sections", []))
figures_html = _render_figures(doc.get("figures", []))
tables_html = _render_tables(doc.get("tables", []))
references_html = _render_references(doc.get("references", []))
sidebar_html = _render_sidebar(doc)

abstract_section = (
  f"""
  <section class=\"surface abstract\">
    <h2>Resumen</h2>
    {abstract_html or '<p>No se proporcionó un resumen.</p>'}
  </section>
  """
)
```

Código Fuente - Transformador XML JATS

```
    ""
)

main_sections = (
    f"""
    <section class=\"article-body\">
        <div class=\"section-grid\">
            {sections_html or '<div class=\"surface article-section empty-state\"><p>Este artículo no contiene
secciones principales.</p></div>'}
        </div>
    </section>
    """
)

figures_section = figures_html or ""
tables_section = tables_html or ""
references_section = references_html or ""

doi_value = doc.get("doi") or ""
meta_values = [
    ("Fecha de publicación", escape(doc.get("pub_date") or "Pendiente")),
    #("Volumen", escape(doc.get("volume") or "N/D")),
    #("Número", escape(doc.get("issue") or "N/D")),
    #(
        #    "Páginas",
        #    escape(" - ".join(filter(None, [doc.get("fpage"), doc.get("lpage")]))) or "N/D",
        #),
    (
        "DOI",
        (
            f'<a href="https://doi.org/{escape(doi_value)}">{escape(doi_value)}</a>'
            if doi_value
            else "N/D"
        ),
    ),
]

meta_grid_html = "".join(
    f'<div class="item"><span class="label">{label}</span>{value}</div>'
    for label, value in meta_values
)

html = f"""<!DOCTYPE html>
<html lang=\"es\">
<head>
    <meta charset=\"utf-8\">
    <meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">
    <title>{title}</title>
    <style>{css}</style>
</head>
<body data-theme=\"light\" data-contrast=\"normal\">
    <div class="app">
        <header class="app-header">
            <div class="header-panel">
                <div class="branding">
                    <h1>{title}</h1>
                    {f'<p class="translated-title">{translated_title}</p>' if translated_title else ''}
                </div>
                <div class="control-bar">
                    <button id="sidebarToggle" type="button" class="sidebar-toggle" aria-label="Mostrar índice"
aria-controls="sidebar" aria-expanded="false">
                        <span>Índice</span>
                    </button>
```


Código Fuente - Transformador XML JATS

```
<button id="themeToggle" aria-label="Cambiar tema"><span>Modo</span><span class="state">Modo
oscuro</span></button>
    <button id="contrastToggle" aria-label="Cambiar contraste"><span>Contraste</span><span
class="state">Alto contraste</span></button>
</div>
</div>
</header>
<div class="layout">
    {sidebar_html}
    <div class="sidebar-backdrop" id="sidebarBackdrop" aria-hidden="true"></div>
    <main class="main">
        <section class="surface metadata">
            <h2>Información general</h2>
            <div class="meta-grid">
                {meta_grid_html}
            </div>
        </section>
        {authors_html}
        {abstract_section}
        {main_sections}
        {figures_section}
        {tables_section}
        {references_section}
    </main>
</div>
    <button id="scrollTopButton" class="floating-button floating-top" type="button" aria-label="Volver al
inicio">
        <span class="icon" aria-hidden="true">&uarr;</span>
        <span class="label">Arriba</span>
    </button>
    <button id="floatingMenuButton" class="floating-button floating-menu" type="button" aria-label="Mostrar
índice flotante" aria-controls="sidebar" aria-expanded="false">
        <span class="icon" aria-hidden="true">&#9776;</span>
        <span class="label">Índice</span>
    </button>
</div>
<script>{js}</script>
</body>
</html>
"""
    return html
```

```
def convert(xml_path: str, output_path: str | None = None) -> str:
    """Función de alto nivel para convertir un archivo JATS XML a HTML.
```

Args:

xml_path (str): Ruta al archivo de entrada.

output_path (str | None, optional): Ruta de salida. Si es None, no guarda archivo.

Returns:

str: El contenido HTML generado.

"""

```
doc = parse_jats(xml_path)
```

```
html = build_html(doc)
```

```
if output_path:
```

```
    Path(output_path).write_text(html, encoding="utf-8")
```

```
return html
```

Código Fuente - Transformador XML JATS

```
def main(argv: List[str]) -> int:
    """Punto de entrada para la ejecución desde línea de comandos.

    Args:
        argv (List[str]): Argumentos de la línea de comandos (sin incluir el nombre del script).

    Returns:
        int: Código de salida (0 para éxito, 1 para error).
    """
    parser = argparse.ArgumentParser(description="Convierte un artículo JATS en HTML estilizado y responsivo.")
    parser.add_argument("xml", help="Ruta al archivo XML en formato JATS")
    parser.add_argument("-o", "--output", help="Ruta del archivo HTML resultante")
    parser.add_argument("--json", action="store_true", help="Mostrar datos extraídos en JSON en lugar de HTML")
    args = parser.parse_args(argv)

    try:
        data = parse_jats(args.xml)
    except Exception as exc: # noqa: BLE001
        print(f"Error al analizar el fichero JATS: {exc}", file=sys.stderr)
        return 1

    if args.json:
        print(json.dumps(data, indent=2, ensure_ascii=False))
        return 0

    html = build_html(data)
    if args.output:
        Path(args.output).write_text(html, encoding="utf-8")
    else:
        print(html)
    return 0

if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1:]))
```

Código Fuente - Transformador XML JATS

Archivo: streamlit_app.py

```
import streamlit as st

# Configuración global de la página
st.set_page_config(
    page_title="Transformador XML JATS",
    page_icon="?",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Definición de las páginas
pages = {
    "Aplicación": [
        st.Page("views/transformador.py", title="Transformador", icon="?"),
    ],
    "Información": [
        st.Page("views/manual_usuario.py", title="Manual de Usuario", icon="?"),
        st.Page("views/documentacion.py", title="Documentación", icon="?"),
        st.Page("views/creditos.py", title="Créditos y Licencias", icon="??"),
    ]
}

# Configuración de navegación
pg = st.navigation(pages)
pg.run()
```

Código Fuente - Transformador XML JATS

Archivo: views/creditos.py

```
import streamlit as st
import base64
from pathlib import Path

# Nota: st.set_page_config removido, se maneja en streamlit_app.py

def _get_logo_base64() -> str:
    """Lee el archivo de logo y lo convierte a Base64."""
    try:
        logo_path = Path("resources/UV_blanco.png")
        if logo_path.exists():
            encoded = base64.b64encode(logo_path.read_bytes()).decode("utf-8")
            return f"data:image/png;base64,{encoded}"
    except Exception:
        pass
    return ""

def main():
    st.title("?? Información del Proyecto")

    col1, col2 = st.columns([1, 2])

    with col1:
        logo_src = _get_logo_base64()
        if logo_src:
            st.markdown(
                f"""
                <div style="text-align: left; margin-bottom: 20px;">
                    
                </div>
                """,
                unsafe_allow_html=True
            )

        st.markdown("""
        ### Universidad de Valparaíso
        **Facultad de Medicina**
        *Escuela de Obstetricia y Puericultura*
        """)

    with col2:
        st.header("Créditos y Desarrollo")
        st.markdown("""
        Esta herramienta ha sido desarrollada para optimizar el flujo de trabajo editorial de la **Revistas
        UV**, automatizando la conversión de manuscritos a XML JATS validado.

        **Desarrollador Principal:**
        Cristian Carreño León
        [cristian.carreno@uv.cl](mailto:cristian.carreno@uv.cl)
        """)

        st.divider()

        st.header("Tecnología")
        st.markdown("""
        El sistema utiliza tecnologías de vanguardia para el procesamiento de texto e inteligencia artificial:

        - **Python 3.9+**: Lenguaje base.
        - **Streamlit**: Framework de interfaz de usuario.
        """)
```

Código Fuente - Transformador XML JATS

```
- **Google Gemini Pro**: Modelo de lenguaje (LLM) para la interpretación semántica y corrección de XML.
- **LXML**: Procesamiento y validación robusta de XML.
- **JATS 1.3**: Estándar de etiquetado (Journal Archiving and Interchange Tag Suite).
""")

st.divider()

st.header("? Privacidad y Tratamiento de Datos")
st.info("""
Importante: Esta aplicación procesa documentos utilizando servicios de terceros.
""")

st.markdown("""
1. Procesamiento Volátil: Los archivos cargados se procesan en la memoria del servidor (o localmente si se ejecuta en su máquina) y no se almacenan permanentemente.
2. API de Inteligencia Artificial:
    - El texto extraído de los documentos se envía a la API de Google Gemini para su estructuración.
      - Google utiliza estos datos de acuerdo con sus [Términos de Servicio de API Generativa](https://ai.google.dev/terms).
      - No suba documentos con datos personales sensibles (nombres de pacientes, datos confidenciales no anonimizados) a menos que tenga autorización explícita.
3. Uso Local Recomendado: Para máxima privacidad, ejecute esta herramienta en un entorno local seguro.
""")

st.divider()

# Botón Volver (más claro)
col_back, _ = st.columns([1, 2])
with col_back:
    if st.button("?? VOLVER AL INICIO", type="primary", use_container_width=True):
        st.switch_page("views/transformador.py")

# Sidebar Footer (Igual que en la app principal)
with st.sidebar:
    st.markdown("---")
    st.markdown(
        """
        <div class='branding'>
            <b>Universidad de Valparaíso</b><br>
            <small>Transformador XML JATS v0.5 (Beta)</small>
        </div>
        """,
        unsafe_allow_html=True
    )

st.caption("Transformador XML JATS v0.5 (Beta) | Universidad de Valparaíso")

main()
```

Código Fuente - Transformador XML JATS

Archivo: views/documentacion.py

```
import streamlit as st
from pathlib import Path

def main():
    st.title("? Documentación del Proyecto")

    tab1, tab2 = st.tabs(["? README", "? CONTRIBUTING"])

    with tab1:
        readme_path = Path("README.md")
        if readme_path.exists():
            st.markdown(readme_path.read_text(encoding='utf-8'))
        else:
            st.warning("README.md no encontrado.")

    with tab2:
        contrib_path = Path("CONTRIBUTING.md")
        if contrib_path.exists():
            st.markdown(contrib_path.read_text(encoding='utf-8'))
        else:
            st.warning("CONTRIBUTING.md no encontrado.")

    # Sidebar Footer (Igual que en la app principal para consistencia)
    with st.sidebar:
        #st.markdown("---")
        st.markdown(
            """
            <div class='branding'>
                <b>Universidad de Valparaíso</b><br>
                <small>Transformador XML JATS v0.5 (Beta)</small>
            </div>
            """,
            unsafe_allow_html=True
        )

main()
```

Código Fuente - Transformador XML JATS

Archivo: views/manual_usuario.py

```
import streamlit as st
import base64
from fpdf import FPDF
import tempfile
import os
from pathlib import Path
from datetime import datetime

# --- CONTENIDO MANUAL DE USUARIO (Hardcoded + Images) ---
MANUAL_SECTIONS = [
    {
        "type": "text",
        "title": "Introducción",
        "content": """
El Transformador XML JATS es una herramienta especializada diseñada para optimizar y automatizar el flujo
de trabajo editorial de la Universidad de Valparaíso.

Esta solución permite la conversión de manuscritos originales en formato Microsoft Word (`.docx`) al estándar
JATS XML (Journal Archiving and Interchange Tag Suite), asegurando el cumplimiento con los requisitos de
indexación y preservación digital de alto nivel.
"""
    },
    {
        "type": "step",
        "title": "1. Carga de Archivos",
        "content": """
- Navegue a la sección "Transformador" en el menú lateral.
- Encontrará un área designada para la carga de archivos.
- Puede arrastrar y soltar su archivo `.docx` o hacer clic en "Browse files" para seleccionarlo de su equipo.

Una vez cargado el archivo, el sistema realizará una extracción automática del texto:

- Revise que el texto extraído sea correcto.
- Si no es correcto, puede editar el texto manualmente.
- Agregue en esta sección información que puede estar ausente en el documento original (DOI, Fecha de
publicación, etc).
- Revise que la información sea correcta.
"""
    },
    {
        "type": "step",
        "title": "2. Generación de XML JATS",
        "content": """
Una vez cargado el archivo, en la pestaña "Generación":

- Presione el botón "Generar XML JATS".
- El sistema procesará el contenido aplicando las reglas del estándar JATS 1.3.
"""
    },
    {
        "type": "step",
        "title": "3. Validación y Corrección",
        "content": """
- Presione el botón "Ejecutar Validación DTD".
- El sistema procesará el contenido aplicando las reglas del estándar JATS 1.3.
"""
    }
]
```

Código Fuente - Transformador XML JATS

- Se ejecutará una validación automática estricta contra el DTD oficial.

Importante

- Si el sistema encuentra errores, se mostrará un informe con los errores encontrados y se habilitarán un botón de corrección que permitirá corregir los errores encontrados asistido por la IA de Gemini.

- Si el sistema no encuentra errores, se mostrará un mensaje de éxito y se habilitarán el botón de Resultados.

```
"""
    "image": "resources/manual_images/03.png", # Reusing home as it shows the button usually
    "caption": "Figura 3: Validación de DTD."
},
{
    "type": "step",
    "title": "4. Resultados",
    "content": ""
```

Si la validación es exitosa, se habilitarán los botones de descarga:

- Descargar XML: Archivo listo para publicación/preservación.

- Descargar HTML: Vista previa para lectura web.

```
"""
    "image": "resources/manual_images/04.png",
    "caption": "Figura 4: Resultados."
}
]
```

FAQ_CONTENT = ""

¿Qué formatos soporta?

Actualmente soporta archivos Microsoft Word (`.docx`). Asegúrese de que el documento no esté protegido con contraseña.

¿Qué hago si falla la validación?

Revise el mensaje de error. Generalmente se debe a caracteres especiales no soportados o estructuras de documento inusuales (ej. tablas anidadas complejas). Edite el contenido en el paso 2 y reintente.

¿Es seguro subir mis archivos?

Sí, los archivos se procesan temporalmente en la memoria para su transformación y no se guardan en el servidor de forma permanente.

"""

```
def get_documentation_content():
```

```
    """Lee README y CONTRIBUTING."""
```

```
    docs = []
```

```
    # README
```

```
    readme_path = Path("README.md")
```

```
    if readme_path.exists():
```

```
        docs.append({"title": "Documentación General (README)", "content":
```

```
readme_path.read_text(encoding='utf-8')})
```

```
    # CONTRIBUTING
```

```
    contrib_path = Path("CONTRIBUTING.md")
```

```
    if contrib_path.exists():
```

```
        docs.append({"title": "Guía de Contribución", "content": contrib_path.read_text(encoding='utf-8')})
```

```
    return docs
```

```
def get_credits_content():
```

```
    """Retorna contenido de créditos como texto."""
```

```
    return ""
```

Esta herramienta ha sido desarrollada para optimizar el flujo de trabajo editorial de la Revistas UV, automatizando la conversión de manuscritos a XML JATS validado.

Desarrollador Principal:

Código Fuente - Transformador XML JATS

Cristian Carreño León (cristian.carreno@uv.cl)

Tecnología:

- Python 3.9+: Lenguaje base.
- Streamlit: Framework de interfaz de usuario.
- Google Gemini Pro: Modelo de lenguaje (LLM).
- LXML: Procesamiento y validación robusta de XML.
- JATS 1.3: Estándar de etiquetado.

Privacidad y Tratamiento de Datos:

1. Procesamiento Volátil: Los archivos no se almacenan permanentemente.
 2. API de Inteligencia Artificial: Se utiliza Google Gemini bajo sus términos de servicio.
- """

```
class ProfessionalPDF(FPDF):
    def header(self):
        # Logo
        logo_path = "resources/UV_color.png"
        if os.path.exists(logo_path):
            self.image(logo_path, 10, 8, 33)

        self.set_font('Arial', 'B', 10)
        self.cell(80) # Move to right
        self.cell(100, 10, 'Manual de Usuario - Transformador XML JATS', 0, 0, 'R')
        self.ln(20)
        # Line break
        self.line(10, 25, 200, 25)

    def footer(self):
        self.set_y(-15)
        self.set_font('Arial', 'I', 8)
        self.set_text_color(128)
        self.cell(0, 10, f'Generado el {datetime.now().strftime("%d/%m/%Y")} | Página ' + str(self.page_no()) +
        '/68', 0, 0, 'C')

    def clean_text_for_pdf(text):
        """Limpia caracteres markdown básicos y elimina caracteres no soportados por latin-1 (emojis)."""
        # Remove bold/italic markers only if they are not part of a header structure we parse later?
        # Actually for simple FPDF text, we want to strip them usually.
        # But let's leave proper headers (#) alone here so we can detect them in the loop.
        text = text.replace('**', '').replace('_', '').replace(`', '').strip()
        return text.encode('latin-1', 'ignore').decode('latin-1')

    def add_markdown_section_to_pdf(pdf, text):
        """Parsea texto markdown simple y lo agrega al PDF formateado."""
        lines = text.split('\n')
        for line in lines:
            line = line.strip()
            if not line:
                pdf.ln(5)
                continue

            # Headers
            if line.startswith('# '):
                pdf.ln(5)
                pdf.set_font('Arial', 'B', 16)
                pdf.set_text_color(0, 51, 102) # Dark Blue
                pdf.cell(0, 10, clean_text_for_pdf(line.replace('# ', '')), 0, 1, 'L')
                pdf.set_text_color(0)
                pdf.set_font('Arial', '', 11)

            elif line.startswith('## '):
```

Código Fuente - Transformador XML JATS

```
pdf.ln(4)
pdf.set_font('Arial', 'B', 14)
pdf.set_text_color(0, 51, 102)
pdf.cell(0, 10, clean_text_for_pdf(line.replace('## ', '')), 0, 1, 'L')
pdf.set_text_color(0)
pdf.set_font('Arial', '', 11)

elif line.startswith('### '):
    pdf.ln(2)
    pdf.set_font('Arial', 'B', 12)
    pdf.cell(0, 10, clean_text_for_pdf(line.replace('### ', '')), 0, 1, 'L')
    pdf.set_font('Arial', '', 11)

# Lists
elif line.startswith('- ') or line.startswith('* '):
    pdf.set_font('Arial', '', 11)
    pdf.cell(5) # Indent
    # chr(149) is bullet for latin-1
    content = clean_text_for_pdf(line[2:])
    pdf.multi_cell(0, 6, chr(149) + ' ' + content)

# Numbered Lists (Simple detection "1. ")
elif len(line) > 2 and line[0].isdigit() and line[1] == '.' and line[2] == ' ':
    pdf.set_font('Arial', '', 11)
    pdf.cell(5) # Indent
    content = clean_text_for_pdf(line)
    pdf.multi_cell(0, 6, content)

# Code blocks (simple detection)
elif line.startswith('``'):
    continue # Skip the marker

# Normal text
else:
    pdf.set_font('Arial', '', 11)
    pdf.multi_cell(0, 6, clean_text_for_pdf(line))

def create_professional_pdf():
    pdf = ProfessionalPDF()
    pdf.alias_nb_pages()

    # --- PORTADA ---
    pdf.add_page()

    # Logo Grande
    logo_path = "resources/UV_color.png"
    if os.path.exists(logo_path):
        pdf.image(logo_path, x=65, y=60, w=80)

    pdf.set_y(100)
    pdf.set_font('Arial', 'B', 24)
    pdf.cell(0, 10, 'Manual de Usuario', 0, 1, 'C')
    pdf.ln(5)
    pdf.set_font('Arial', '', 16)
    pdf.set_text_color(100)
    pdf.cell(0, 10, 'Transformador XML-JATS', 0, 1, 'C')

    pdf.set_y(130)
    pdf.set_font('Arial', '', 12)
    pdf.set_text_color(0)
    pdf.cell(0, 10, 'Universidad de Valparaíso', 0, 1, 'C')
    pdf.cell(0, 10, 'Facultad de Medicina', 0, 1, 'C')
```

Código Fuente - Transformador XML JATS

```
pdf.set_y(180)
pdf.set_font('Arial', 'B', 11)
pdf.cell(0, 10, 'Autor: Cristian Carreño León', 0, 1, 'C')
pdf.cell(0, 10, 'Fecha: 11-12-2025', 0, 1, 'C')
pdf.cell(0, 10, 'Versión: 0.5 (Beta)', 0, 1, 'C')
pdf.cell(0, 10, 'Contacto: carreonleong@gmail.com', 0, 1, 'C')

# --- SECCIÓN 1: DOCUMENTACIÓN ---
docs = get_documentation_content()
for doc in docs:
    pdf.add_page()
    # Title of the section (e.g. "Documentación General")
    pdf.set_font('Arial', 'B', 16)
    pdf.set_fill_color(240, 240, 240)
    pdf.cell(0, 10, doc["title"], 0, 1, 'L', 1)
    pdf.ln(5)

    # Render Markdown content properly
    add_markdown_section_to_pdf(pdf, doc["content"])

# --- SECCIÓN 2: MANUAL DE USUARIO ---
pdf.add_page()

# Intro
pdf.set_font('Arial', 'B', 16)
pdf.set_fill_color(240, 240, 240)
pdf.cell(0, 10, 'Manual de Usuario', 0, 1, 'L', 1)
pdf.ln(5)

pdf.set_font('Arial', '', 11)
intro_text = clean_text_for_pdf(MANUAL_SECTIONS[0]["content"])
pdf.multi_cell(0, 6, intro_text)
pdf.ln(10)

# Pasos
pdf.set_font('Arial', 'B', 14)
pdf.cell(0, 10, 'Flujo de Trabajo', 0, 1, 'L')
pdf.ln(5)

for section in MANUAL_SECTIONS:
    if section["type"] == "step":
        # Title
        pdf.set_font('Arial', 'B', 13)
        pdf.set_text_color(0, 51, 102) # Dark Blue
        pdf.cell(0, 10, section["title"], 0, 1, 'L')
        pdf.set_text_color(0)

        # Content (using markdown parser logic slightly adapted or reuse)
        # The manual sections are simple, mostly lines. Let's use clean_text_for_pdf logic directly
        # but allow bullet points if any exist in the manual content.
        # Actually, let's use the new parser for consistency!
        add_markdown_section_to_pdf(pdf, section["content"])
        pdf.ln(2)

        # Image
        if "image" in section and os.path.exists(section["image"]):
            pdf.image(section["image"], w=140, x=35)
            if "caption" in section:
                pdf.set_font('Arial', 'I', 9)
                pdf.cell(0, 8, section["caption"], 0, 1, 'C')
```

Código Fuente - Transformador XML JATS

```
pdf.ln(10)

# FAQ
pdf.add_page()
pdf.set_font('Arial', 'B', 14)
pdf.cell(0, 10, 'Preguntas Frecuentes', 0, 1, 'L')
pdf.ln(5)

# Use parser for FAQ too to handle bold headers if they were properly markdown
# (Currently FAQ_CONTENT uses ** for bold which clean_text strips,
# but we can improve FAQ_CONTENT to use ## if we want headers, or just rely on the existing logic there).
# Since FAQ_CONTENT uses simple **Question?** format, the previous logic was fine.
# Let's clean it up to use standard markdown ## for questions if possible,
# or just stick to the manual parsing for FAQ which was working okay.
# The previous code handled "?" detection. Let's keep a simple loop for FAQ or adapt it.

faq_lines = FAQ_CONTENT.strip().split('\n')
for line in faq_lines:
    line = clean_text_for_pdf(line)
    if not line:
        pdf.ln(5)
        continue
    if "?" in line and line.endswith("?"):
        pdf.set_font('Arial', 'B', 11)
        pdf.multi_cell(0, 6, line)
        pdf.set_font('Arial', '', 11)
    else:
        pdf.multi_cell(0, 6, line)

# --- SECCIÓN 3: CRÉDITOS Y LICENCIAS ---
pdf.add_page()
pdf.set_font('Arial', 'B', 16)
pdf.set_fill_color(240, 240, 240)
pdf.cell(0, 10, 'Créditos y Licencias', 0, 1, 'L', 1)
pdf.ln(10)

pdf.set_font('Arial', '', 11)
# Using parser for credits? Credits is simple text.
credits_text = get_credits_content()
add_markdown_section_to_pdf(pdf, credits_text)

return pdf

@st.cache_data(show_spinner="Generando PDF...")
def generate_pdf_bytes():
    """Genera el PDF y devuelve los bytes, cacheado para optimizar."""
    try:
        pdf = create_professional_pdf()
        # Use temp file to get bytes safely across platforms/versions
        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp_file:
            pdf.output(tmp_file.name)
            tmp_path = tmp_file.name

        with open(tmp_path, "rb") as f:
            pdf_bytes = f.read()

        os.unlink(tmp_path)
        return pdf_bytes
    except Exception as e:
        st.error(f"Error generando PDF: {e}")
        return None
```

Código Fuente - Transformador XML JATS

```
def main():
    st.title("? Manual de Usuario")
    st.markdown("---")

    # Layout: Intro wide
    st.markdown(MANUAL_SECTIONS[0]["content"])

    st.subheader("? Flujo de Trabajo")

    # Zig-zag layout using loop
    for i, section in enumerate(MANUAL_SECTIONS):
        if section["type"] == "step":
            with st.container():
                col_text, col_img = st.columns([1, 1], gap="large")

                # Alternating layout
                if i % 2 != 0:
                    col_text, col_img = col_img, col_text

                with col_text:
                    st.markdown(f"### {section['title']}")
                    # Use the parser for the web view too?
                    # The content is string with markdown, st.markdown handles it well.
                    # But section content might be cleaner if we use st.markdown directly.
                    st.markdown(section["content"])

                with col_img:
                    if os.path.exists(section["image"]):
                        st.image(section["image"], caption=section["caption"], width=400) # Fixed width for
consistency or use_container_width
                    else:
                        st.warning("Imagen no encontrada")

                st.divider()

    with st.expander("? Preguntas Frecuentes", expanded=False):
        st.markdown(FAQ_CONTENT)

    st.markdown("---")

    col_center = st.columns([1, 2, 1])
    with col_center[1]:
        st.info("Obtenga el manual completo en PDF, incluyendo documentación y créditos.")

    # Pre-generate or get from cache
    pdf_bytes = generate_pdf_bytes()

    if pdf_bytes:
        st.download_button(
            label="? Descargar Manual Completo PDF",
            data=pdf_bytes,
            file_name="Manual_Usuario_Completo_UV.pdf",
            mime="application/pdf",
            type="primary",
            use_container_width=True
        )

if __name__ == "__main__":
    main()
```

Código Fuente - Transformador XML JATS

Archivo: views/transformador.py

```
import streamlit as st
import os
import tempfile
import time
from pathlib import Path
from typing import Optional, Tuple, Dict, Any, List

from modules import transformer
from modules import xml_html
from modules import correction
import streamlit.components.v1 as components

# Nota: st.set_page_config se ha movido a streamlit_app.py

# CSS Personalizado
st.markdown("""
<style>
    .main {
        /* Dejar fondo por defecto */
    }
    .stButton>button {
        width: 100%;
        border-radius: 5px;
        height: 3em;
        background-color: #003366;
        color: white;
        border: none;
    }
    .stButton>button:hover {
        background-color: #002244;
    }
    h1 {
        color: #003366;
    }
    @media (prefers-color-scheme: dark) {
        h1 { color: #8ab4f8; }
    }
    .branding {
        text-align: center;
        margin-top: 20px;
        color: #666;
        font-size: 0.9em;
    }
    .success-nav {
        /* Ya no se usa para bloque estático, pero mantenemos por compat */
        padding: 15px;
        border-radius: 8px;
        background-color: #d4edda;
        color: #155724;
        margin-top: 10px;
        border: 1px solid #c3e6cb;
    }
</style>
""", unsafe_allow_html=True)

# Helper para cambiar Tabs via JS
def js_switch_tab(tab_index: int):
    js_code = f"""
    <script>
```

Código Fuente - Transformador XML JATS

```
        var tabs = window.parent.document.querySelectorAll('button[data-baseweb="tab"]');
        if (tabs[{tab_index}]) {{
            tabs[{tab_index}].click();
        }}
    </script>
    """
    components.html(js_code, height=0, width=0)

def main() -> None:
    """Función principal de la aplicación Streamlit."""
    st.title("? Transformador XML JATS")
    st.markdown("### Convierte documentos Word a XML JATS con IA")

    with st.sidebar:
        st.header("Instrucciones")
        st.info(
            "1. **Carga**: Sube el DOCX y extrae el texto.\n"
            "2. **Generación**: Crea el XML.\n"
            "3. **Validación**: Verifica y corrige errores.\n"
            "4. **Resultados**: Descarga el XML/HTML."
        )

    if 'extracted_text' not in st.session_state:
        st.session_state.extracted_text = ""
    if 'generated_xml' not in st.session_state:
        st.session_state.generated_xml = ""
    if 'generated_html' not in st.session_state:
        st.session_state.generated_html = ""
    if 'validation_errors' not in st.session_state:
        st.session_state.validation_errors = []
    if 'correction_chat' not in st.session_state:
        st.session_state.correction_chat = []
    if 'show_correction_chat' not in st.session_state:
        st.session_state.show_correction_chat = False
    if 'pending_correction_xml' not in st.session_state:
        st.session_state.pending_correction_xml = None
    if 'uploaded_file_path' not in st.session_state:
        st.session_state.uploaded_file_path = None

    # Definimos Tabs
    # Tab 0: Carga
    # Tab 1: Generación
    # Tab 2: Validación
    # Tab 3: Resultados
    tab1, tab2, tab3, tab4 = st.tabs([
        "1. Carga y Extracción",
        "2. Generación",
        "3. Validación y Corrección",
        "4. Resultados"
    ])

    # --- Tab 1 ---
    with tab1:
        st.header("Carga de Documento")
        uploaded_file = st.file_uploader("Selecciona un documento Word (.docx)", type=['docx'])

        if uploaded_file is not None:
            if st.session_state.uploaded_file_path is None:
                with tempfile.NamedTemporaryFile(delete=False, suffix=".docx") as tmp_file:
                    tmp_file.write(uploaded_file.getvalue())
                    st.session_state.uploaded_file_path = tmp_file.name
```

Código Fuente - Transformador XML JATS

```
col1, col2 = st.columns([1, 2])
with col1:
    st.success("Archivo cargado.")
    if st.button("Extraer Contenido", type="primary"):
        with st.spinner("Procesando documento..."):
            transformer.extraer_contenido_estructurado(st.session_state.uploaded_file_path)
            if extracted:
                st.session_state.extracted_text = extracted
                st.toast("Extracción exitosa", icon="👉")
            else:
                st.error("Error en la extracción.")

    # Nav Button
    if st.session_state.extracted_text:
        st.success("👉 Texto extraído correctamente.")
        if st.button("👉 Ir a Paso 2: Generación"):
            js_switch_tab(1)

with col2:
    if st.session_state.extracted_text:
        st.text_area("Texto Extraído", value=st.session_state.extracted_text, height=400)

# --- Tab 2 ---
with tab2:
    st.header("Generación XML con IA")
    if not st.session_state.extracted_text:
        st.info("👉 Completa el Paso 1 primero.")
    else:
        if st.button("Generar XML JATS", type="primary"):
            progress_bar = st.progress(0, text="Iniciando...")
            status_text = st.empty()
            status_text.text("Preparando datos...")
            progress_bar.progress(10)
            time.sleep(0.5)

            status_text.text("Enviando a Gemini AI...")
            prompt = transformer.construir_prompt_avanzado(st.session_state.extracted_text)
            progress_bar.progress(30)

            result = transformer.invocar_gemini_cli(prompt)

            status_text.text("Procesando respuesta...")
            progress_bar.progress(80)

            if result.get('returncode') == 0 and result.get('stdout'):
                xml_out = result.get('stdout', '')
                import re
                match = re.search(r"```\xml\s*(.*?)\s*```", xml_out, re.DOTALL)
                if match:
                    xml_out = match.group(1)

                st.session_state.generated_xml = xml_out.strip()
                st.session_state.validation_errors = []
                st.session_state.correction_chat = []
                st.session_state.show_correction_chat = False
                st.session_state.pending_correction_xml = None

                progress_bar.progress(100)
                status_text.text("¡Completado!")
            else:
                progress_bar.empty()
```


Código Fuente - Transformador XML JATS

```
status_text.error("Falló la generación.")
st.error(result.get('stderr'))

if st.session_state.generated_xml:
    st.success("? XML Generado.")
    st.code(st.session_state.generated_xml, language='xml')
    if st.button("?? Ir a Paso 3: Validación"):
        js_switch_tab(2)

# --- Tab 3 ---
with tab3:
    st.header("Validación y Corrección")
    if not st.session_state.generated_xml:
        st.info("?? Genera el XML en el Paso 2 primero.")
    else:
        col_val, col_chat = st.columns([1, 1], gap="large")

        with col_val:
            st.subheader("Estado de Validación")
            if st.button("Ejecutar Validación DTD"):
                st.session_state.pending_correction_xml = None

                is_valid, errors = transformer.validar_jats_xml(st.session_state.generated_xml)
                if is_valid:
                    st.session_state.validation_errors = []
                    st.session_state.show_correction_chat = False
                    st.success("? XML Válido")
                    # Botón dentro del IF para no confundir
                else:
                    st.session_state.validation_errors = errors
                    st.session_state.show_correction_chat = True
                    st.error(f"? {len(errors)} errores encontrados")

            # Mostrar botón de ir a Resultados SOLO si está válido
            if not st.session_state.validation_errors and st.session_state.generated_xml:
                # Checkeamos si ya se validó (errors vacío, pero generated_xml existe...
                # podría no haberse validado aún, pero asumimos flujo normal)
                # Mejor usar una flag si 'is_validated'
                pass

            if st.session_state.validation_errors:
                for e in st.session_state.validation_errors:
                    st.warning(f"? {e}")

            st.divider()
            if st.button("?? Solucionar con IA", type="primary"):
                st.session_state.show_correction_chat = True
                if not st.session_state.correction_chat:
                    with st.spinner("Analizando errores y buscando soluciones..."):
                        res = correction.analizar_errores_inicial(
                            st.session_state.generated_xml,
                            st.session_state.validation_errors
                        )
                    response_text = ""
                    if res.get('returncode') == 0 and res.get('stdout'):
                        response_text = res.get('stdout', '')
                    else:
                        response_text = f"Error: {res.get('stderr')}}"

                    import re
                    match = re.search(r"```xml\s*(.*?)\s*```", response_text, re.DOTALL)
                    if match:
```

Código Fuente - Transformador XML JATS

```
        new_xml = match.group(1)
        st.session_state.pending_correction_xml = new_xml
        # Limpiamos el texto para mostrar solo la nota explicativa si la hay
        # Opcional: mostrar todo
        st.session_state.correction_chat.append({
            "role": "assistant",
            "content": response_text
        })
        st.rerun()
    else:
        # Si no hay errores, mostramos el botón de avanzar
        st.markdown("---")
        if st.button("?? Ir a Paso 4: Resultados"):
            js_switch_tab(3)

with col_chat:
    if st.session_state.show_correction_chat:
        st.subheader("? Asistente de Corrección")

        chat_container = st.container(height=350)
        with chat_container:
            for msg in st.session_state.correction_chat:
                with st.chat_message(msg["role"]):
                    content = msg["content"]
                    if "`xml" in content and msg["role"] == "assistant":
                        text_part = content.split("`xml")[0]
                        st.write(text_part.strip())
                        st.info("? (He generado un XML corregido. Revisa abajo ?)")
                    else:
                        st.write(content)

        prompt = st.chat_input("Escribe tu instrucción...")
        if prompt:
            st.session_state.correction_chat.append({"role": "user", "content": prompt})
            with chat_container:
                with st.chat_message("user"):
                    st.write(prompt)

                with st.chat_message("assistant"):
                    with st.spinner("Consultando a Gemini..."):
                        res = correction.corregir_xml(
                            st.session_state.generated_xml,
                            st.session_state.validation_errors,
                            prompt
                        )

                    response_text = ""
                    if res.get('returncode') == 0 and res.get('stdout'):
                        response_text = res.get('stdout', '')
                    else:
                        response_text = f"Error: {res.get('stderr')}}"

            import re
            match = re.search(r"`xml\s*(.*?)\s`", response_text, re.DOTALL)
            if match:
                new_xml = match.group(1)
                st.session_state.pending_correction_xml = new_xml
                st.session_state.correction_chat.append({
                    "role": "assistant",
                    "content": response_text
                })
```

Código Fuente - Transformador XML JATS

```
        st.rerun()
    else:
        st.write(response_text)
        st.session_state.correction_chat.append({"role": "assistant",
"content": response_text})

    if st.session_state.pending_correction_xml:
        st.divider()
        st.success("? ¡Corrección Generada!")
        st.markdown("***La IA ha propuesto una nueva versión del XML. Revísala y aplícala si
estás de acuerdo.**")

        cols_btn = st.columns([1, 1])
        with cols_btn[0]:
            if st.button("? Reemplazar XML Original", type="primary"):
                st.session_state.generated_xml =
st.session_state.pending_correction_xml.strip()
                st.session_state.validation_errors = []
                st.session_state.pending_correction_xml = None
                # Limpiar chat para iniciar nueva validación limpia
                st.session_state.correction_chat = []
                st.session_state.show_correction_chat = False

                st.toast("XML Actualizado. Ejecuta validación nuevamente.", icon="?")
                time.sleep(1.5)
                st.rerun()

        with st.expander("Ver Diferencias / Nuevo XML", expanded=False):
            st.code(st.session_state.pending_correction_xml, language='xml')

# --- Tab 4 ---
with tab4:
    st.header("Descargas y HTML")
    if not st.session_state.generated_xml:
        st.info("?? Genera el contenido primero.")
    else:
        c1, c2 = st.columns(2)
        with c1:
            st.download_button(
                "?? Descargar XML",
                st.session_state.generated_xml,
                file_name="articulo.xml",
                mime="application/xml"
            )
        with c2:
            if st.button("Generar HTML"):
                with tempfile.NamedTemporaryFile(delete=False, suffix=".xml", mode='w', encoding='utf-8')
as tmp:
                    tmp.write(st.session_state.generated_xml)
                    tmp_path = tmp.name
                try:
                    data = xml_html.parse_jats(tmp_path)
                    html = xml_html.build_html(data)
                    st.session_state.generated_html = html
                except Exception as e:
                    st.error(f"Error HTML: {e}")

            if st.session_state.generated_html:
                st.download_button(
                    "?? Descargar HTML",
                    st.session_state.generated_html,
                    file_name="articulo.html",
```

Código Fuente - Transformador XML JATS

```
        mime="text/html"
    )

    if st.session_state.generated_html:
        st.markdown("---")
        components.html(st.session_state.generated_html, height=800, scrolling=True)

# Sidebar Footer (Igual que en la app principal para consistencia)
with st.sidebar:
    st.markdown("---")
    st.markdown(
        """
        <div class='branding'>
            <b>Universidad de Valparaíso</b><br>
            <small>Transformador XML JATS v0.5 (Beta)</small>
        </div>
        """,
        unsafe_allow_html=True
    )

main()
```